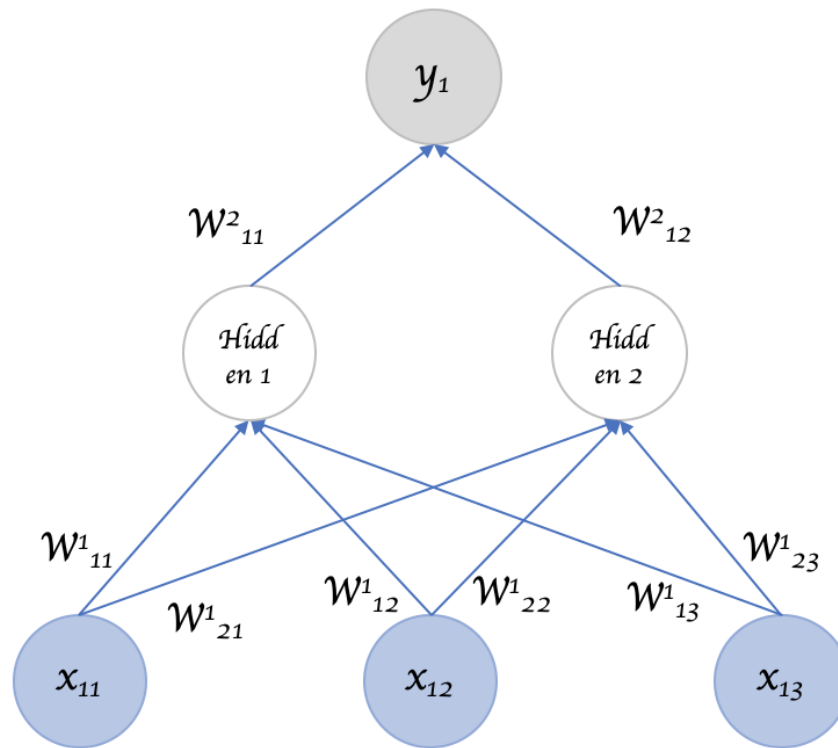


Introduction to Recurrent Neural Network

Motivation for RNN

- the clouds are in the *sky*
 - *Short term dependency can be handled by n grams*
- I grew up in France... I speak fluent *French*
 - *Long term dependency cannot be handled by n grams*

A simple neural network with 1 hidden layer



Issues of ANN with sequence problems

Variable size of input/output neurons

No parameter sharing

Too much computation

Fixed-window

~~as the proctor started the clock~~

discard

the students opened their _____

fixed window

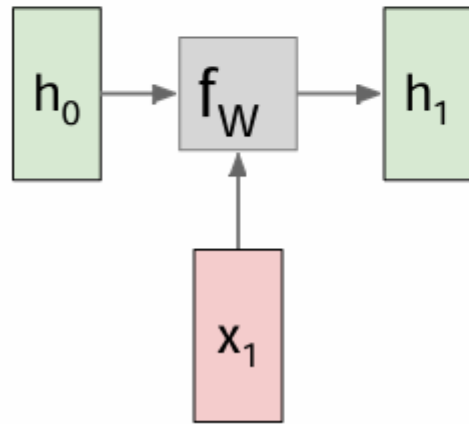
- Language needs context
- This architecture provides context or memory.

Applications of RNN

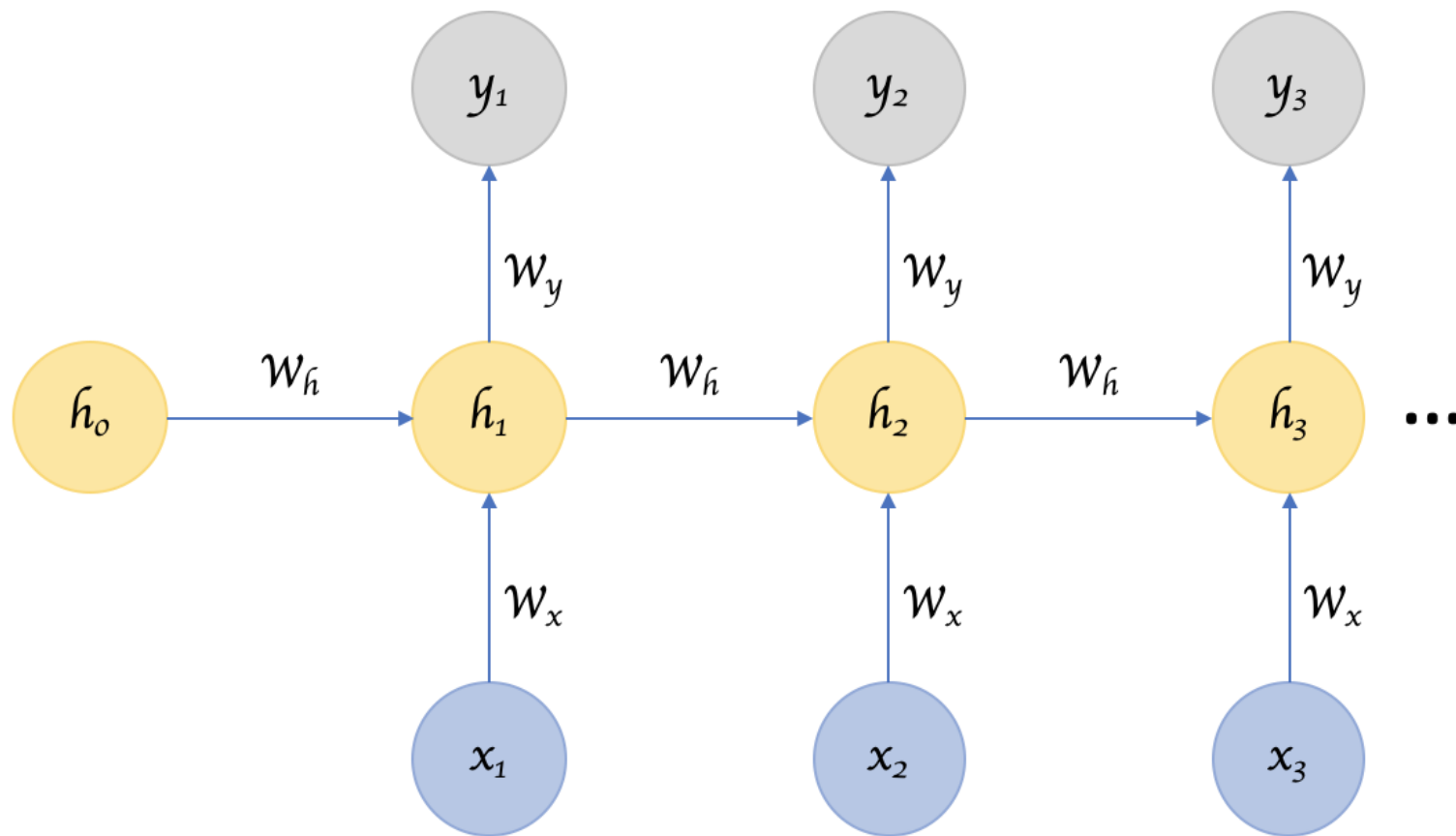
Sequence Data

- Text (sequence of words or sequence of characters)
- Stock prices (sequence of stock prices daily)
- Consumer purchase history
- Many more

RNN: Computational Graph

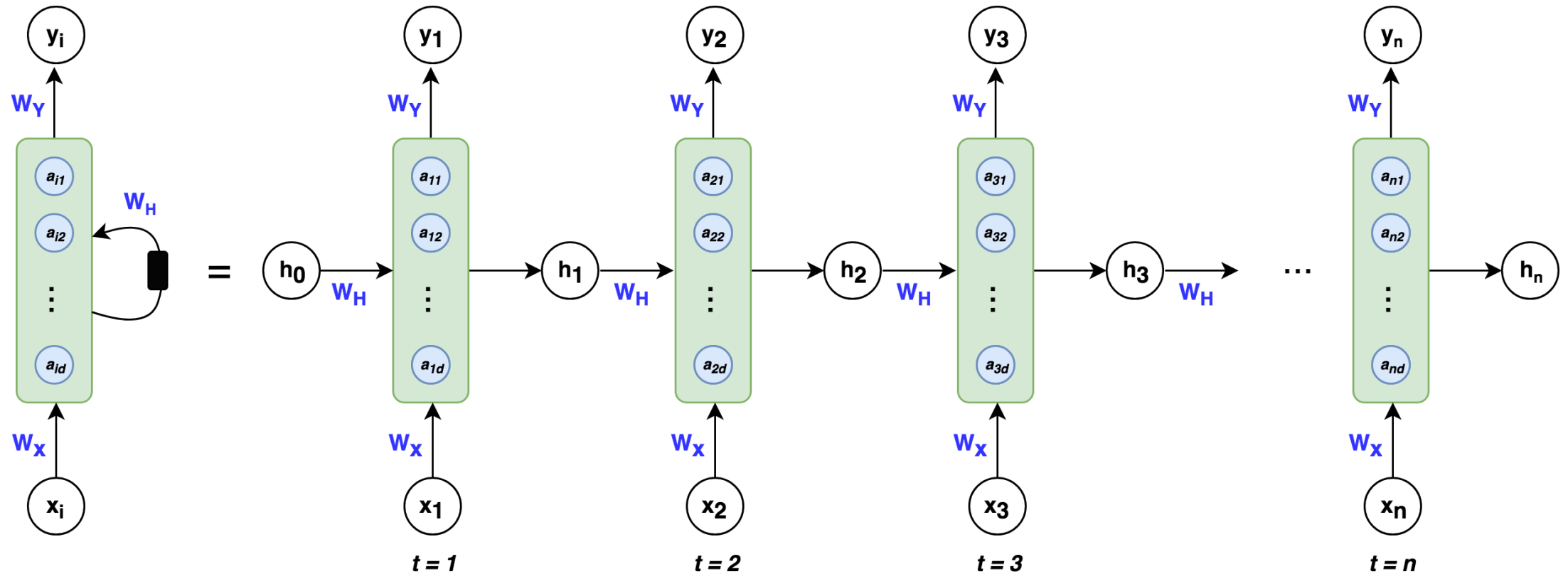


A recurrent neural network with 1 hidden layer



RNN Equations for Forward Pass

Left: Shorthand notation often used for RNNs, **Right:** Unfolded notation for RNNs



One-Hot Encoding of the word “dogs”

$$\mathbf{d} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

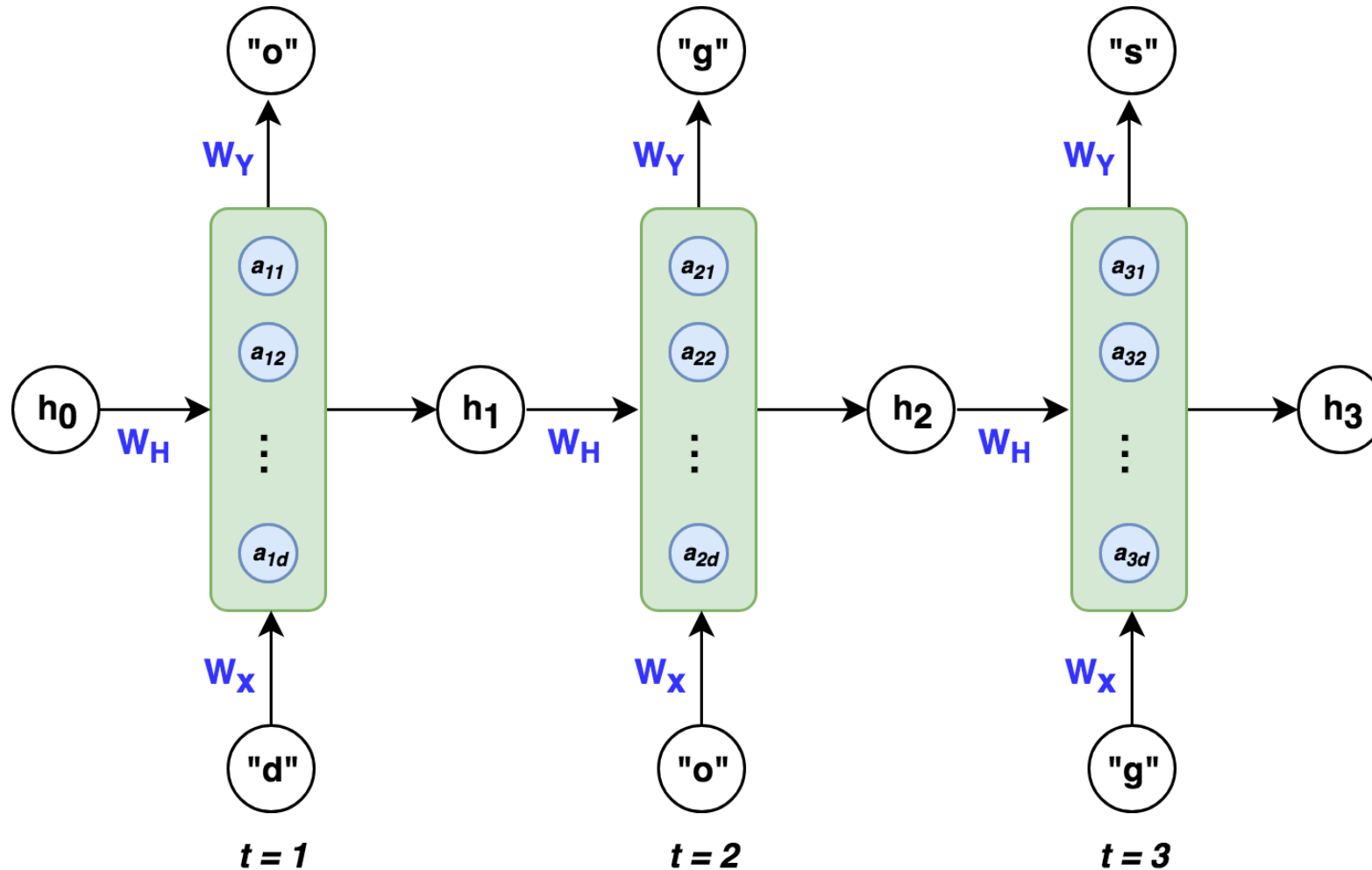
$$\mathbf{o} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\mathbf{g} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\mathbf{s} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

An Example

RNN architecture predicting the letter “s” in “dogs”



RNN Equations

$$a_t = W_H h_{t-1} + W_X X_t \quad \text{Hidden Nodes}$$

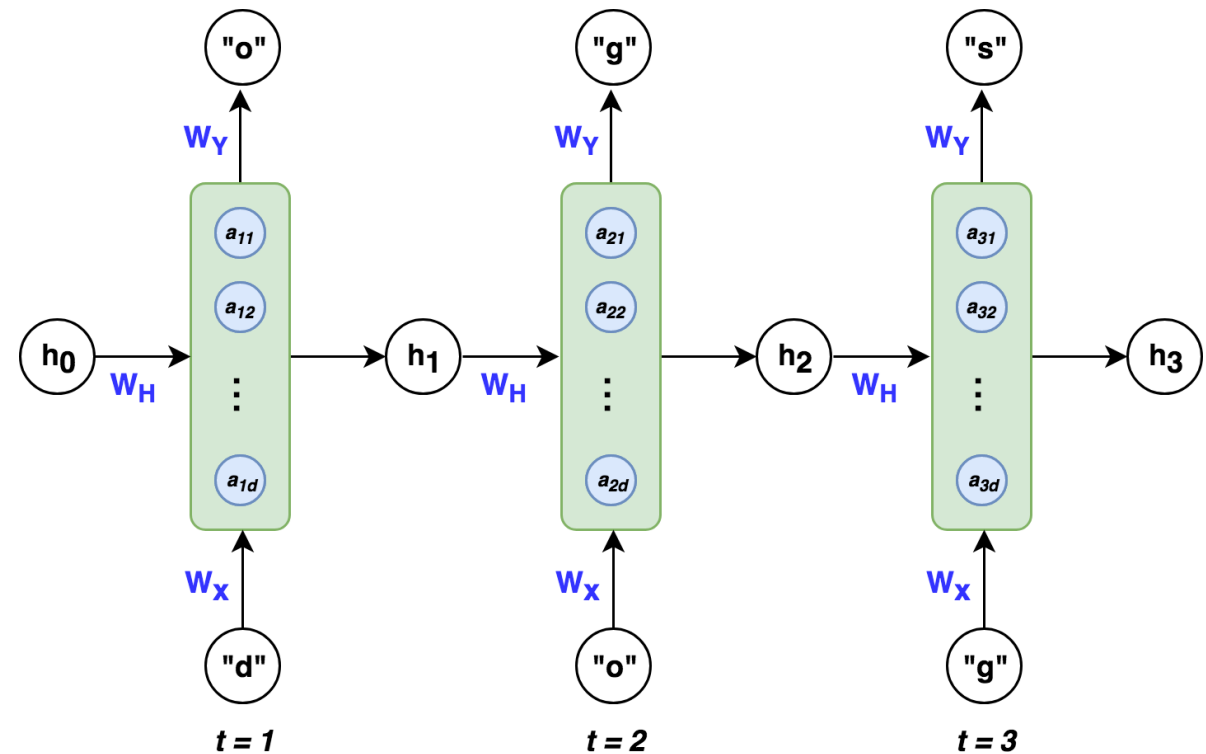
Activation Function

Output
From Hidden
State

$$h_t = \underbrace{\tanh(a_t)}_{\text{Hidden State}}$$

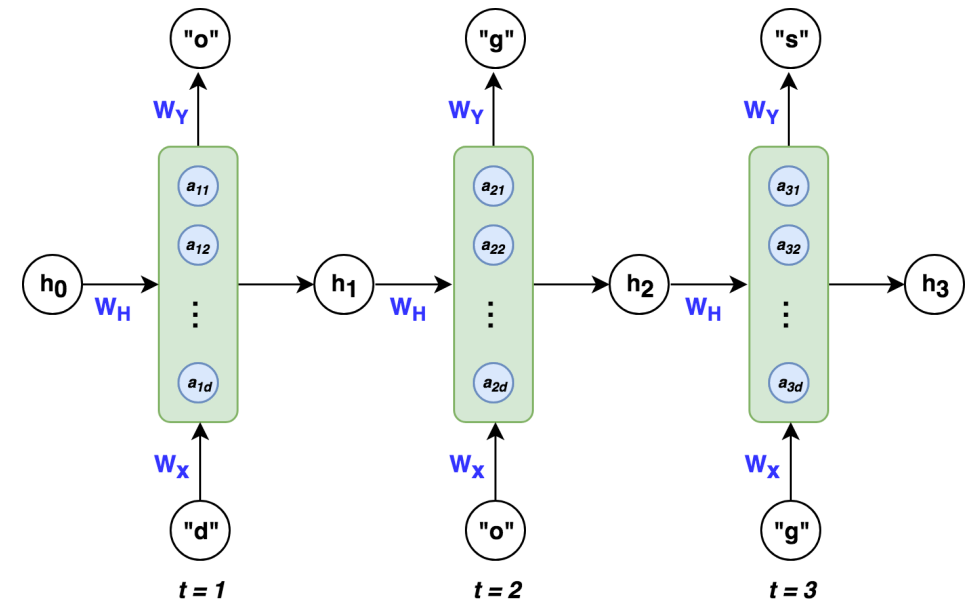
Prediction at
time t

$$y_t = \text{softmax}(W_Y h_t)$$



RNN Equations

$$\begin{array}{c}
 \begin{array}{ccc}
 (d \times d) & (d \times 1) & (k \times 1) \\
 \downarrow & \downarrow & \downarrow \\
 a_t = & \boxed{W_H h_{t-1} + W_X X_t} & \\
 \uparrow & & \\
 (d \times 1) \longrightarrow h_t = \boxed{\tanh(a_t)} & \begin{array}{l} (d \times k) \\ (k \times d) \end{array} & \\
 \downarrow & & \\
 (k \times 1) \longrightarrow y_t = \text{softmax}(W_Y h_t) & &
 \end{array}
 \end{array}$$

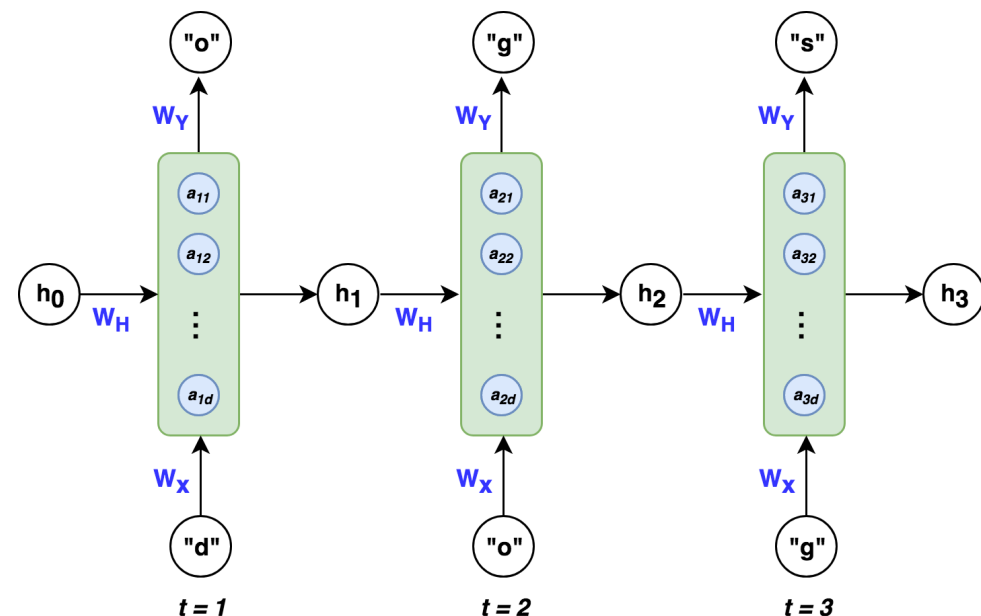
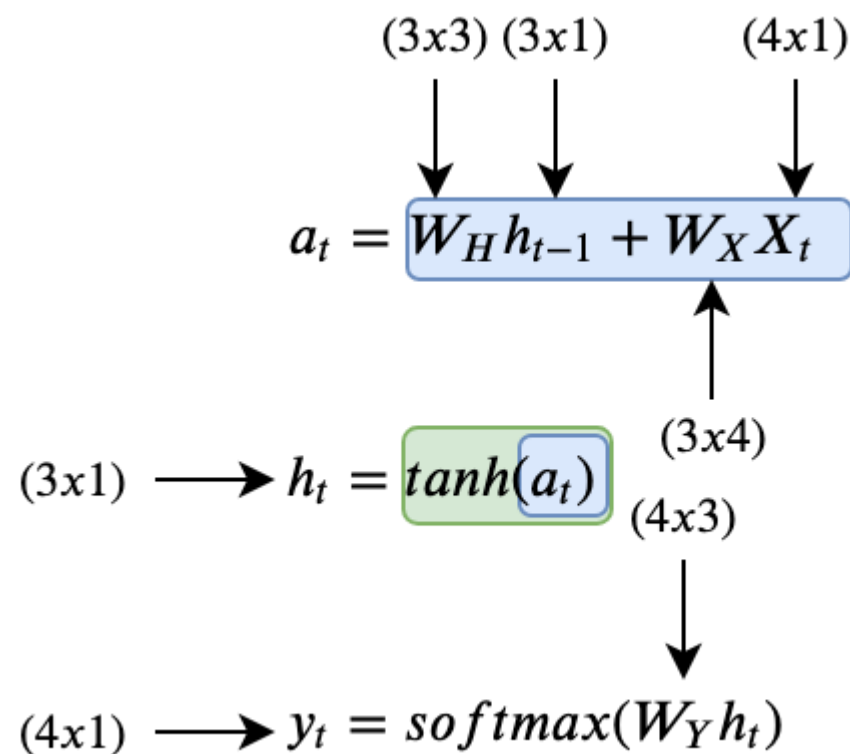


Where

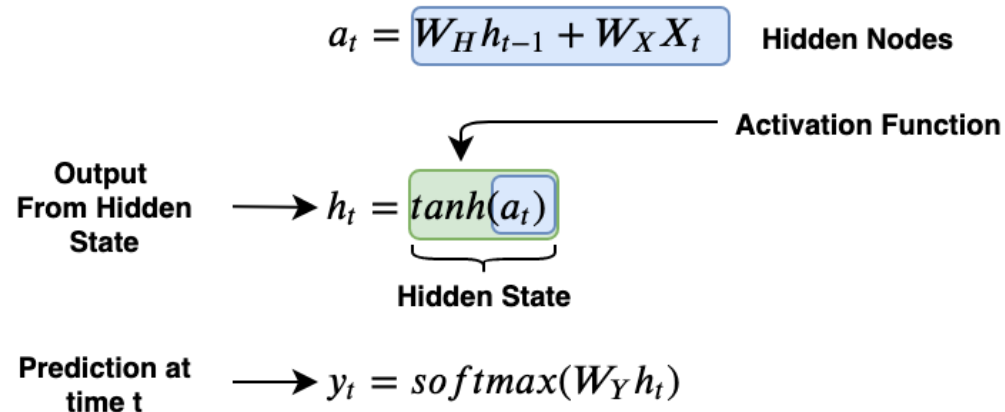
k is the dimension of the input vector x_i

d is the number of hidden nodes

Matrix Dimensions



Forward Propagation



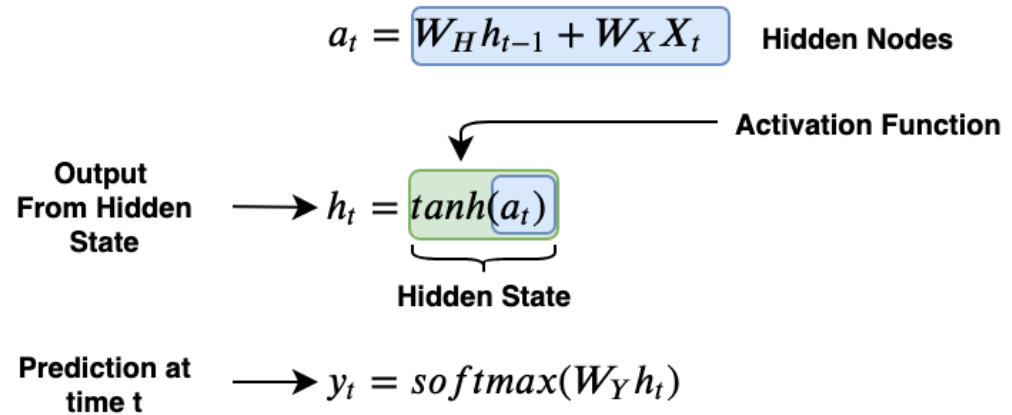
At $t = 1$

$$a_1 = \begin{pmatrix} W_{H,11} & W_{H,12} & W_{H,13} \\ W_{H,21} & W_{H,22} & W_{H,23} \\ W_{H,31} & W_{H,32} & W_{H,33} \end{pmatrix} \begin{pmatrix} h_{0,1} \\ h_{0,2} \\ h_{0,3} \end{pmatrix} + \begin{pmatrix} W_{X,11} & W_{X,12} & W_{X,13} & W_{X,14} \\ W_{X,21} & W_{X,22} & W_{X,23} & W_{X,24} \\ W_{X,31} & W_{X,32} & W_{X,33} & W_{X,34} \end{pmatrix} \begin{pmatrix} x_{1,1} \\ x_{1,2} \\ x_{1,3} \\ x_{1,4} \end{pmatrix} = \begin{pmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \end{pmatrix}$$

$$h_1 = \tanh \left(\begin{pmatrix} a_{1,1} \\ a_{1,2} \\ a_{1,3} \end{pmatrix} \right) = \begin{pmatrix} h_{1,1} \\ h_{1,2} \\ h_{1,3} \end{pmatrix}$$

$$y_1 = \text{softmax} \left(\begin{pmatrix} W_{Y,11} & W_{Y,12} & W_{Y,13} \\ W_{Y,21} & W_{Y,22} & W_{Y,23} \\ W_{Y,31} & W_{Y,32} & W_{Y,33} \\ W_{Y,41} & W_{Y,42} & W_{Y,43} \end{pmatrix} \begin{pmatrix} h_{1,1} \\ h_{1,2} \\ h_{1,3} \end{pmatrix} \right) = \begin{pmatrix} y_{1,1} \\ y_{1,2} \\ y_{1,3} \\ y_{1,4} \end{pmatrix}$$

Forward Propagation

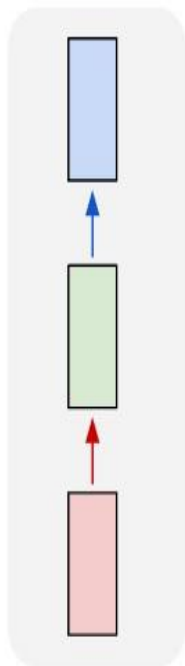


$$a_1 = \begin{pmatrix} 0.1 & 0.5 & 0.1 \\ 0.5 & 0.9 & 0.3 \\ 0.3 & 0.2 & 0.1 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 0.6 & 0.8 & 0.4 & 0.8 \\ 0.2 & 0.2 & 0.8 & 0.7 \\ 0.9 & 0.8 & 0.1 & 0.2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0.2 \\ 0.9 \end{pmatrix}$$

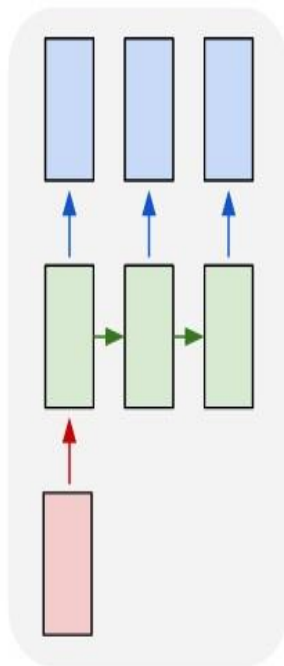
$$h_1 = \tanh\left(\begin{pmatrix} 0.6 \\ 0.2 \\ 0.9 \end{pmatrix}\right) = \begin{pmatrix} 0.54 \\ 0.20 \\ 0.72 \end{pmatrix}$$

$$y_1 = \text{softmax}\left(\begin{pmatrix} 0.9 & 0.8 & 0.3 \\ 0.2 & 0.3 & 0.4 \\ 0.6 & 0.9 & 0.1 \\ 0.5 & 0.0 & 0.3 \end{pmatrix} \begin{pmatrix} 0.54 \\ 0.20 \\ 0.72 \end{pmatrix}\right) = \begin{pmatrix} 0.32 \\ 0.21 \\ 0.24 \\ 0.22 \end{pmatrix}$$

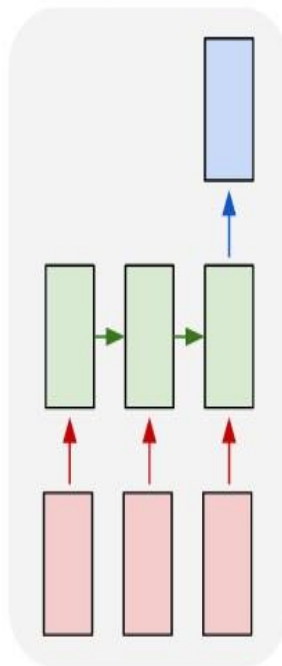
one to one



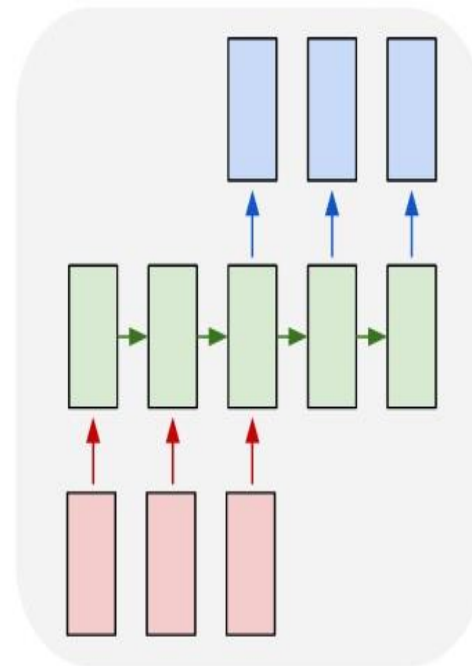
one to many



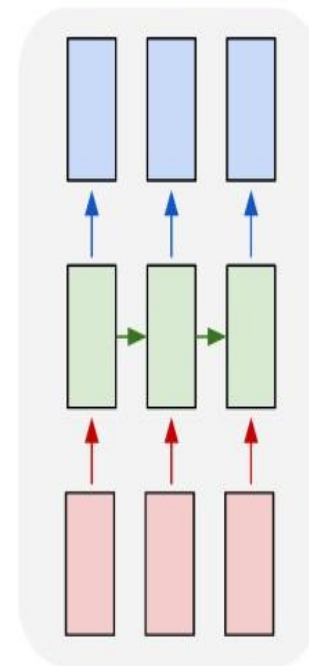
many to one



many to many

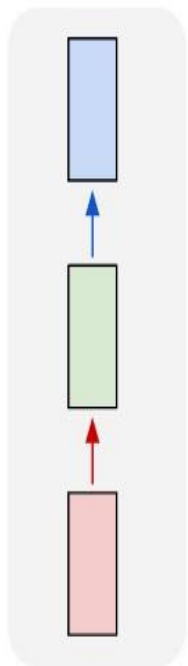


many to many

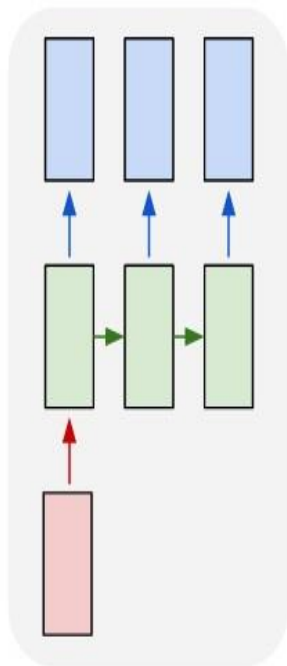


Vanilla Neural Networks

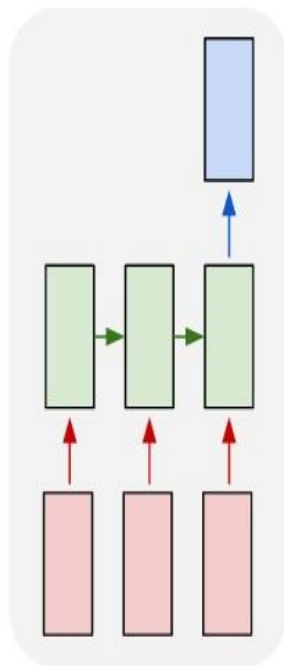
one to one



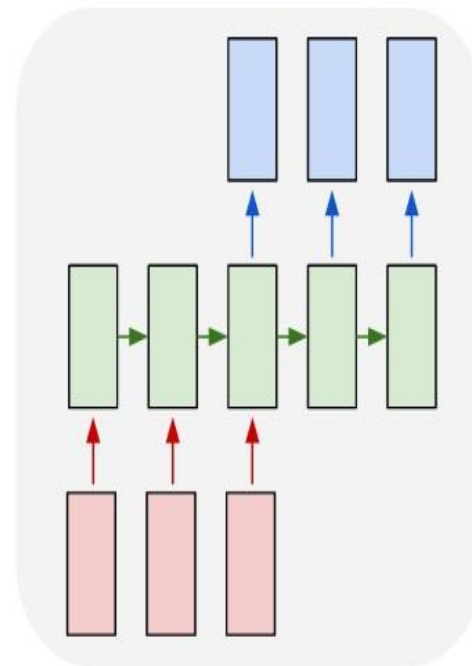
one to many



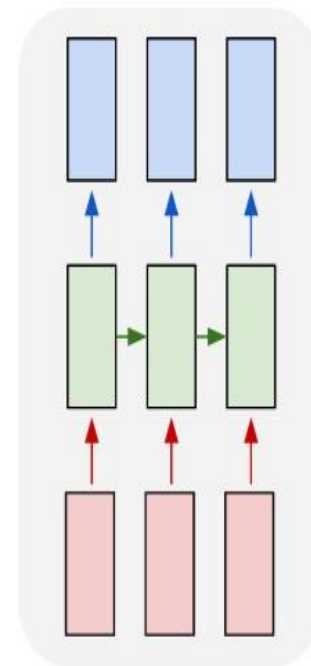
many to one



many to many

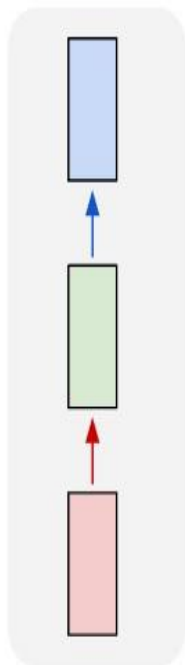


many to many

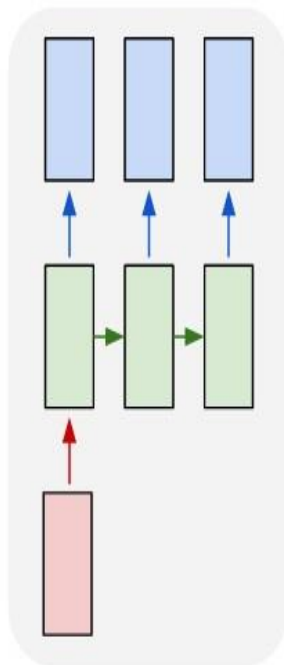


e.g. **Image Captioning**
image -> sequence of words

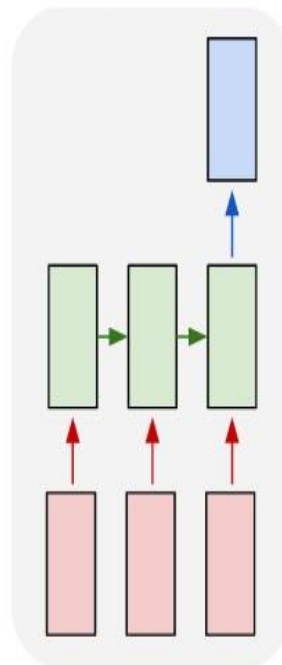
one to one



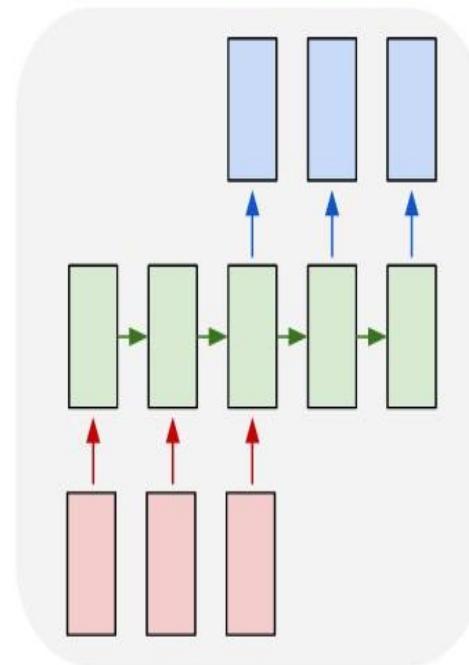
one to many



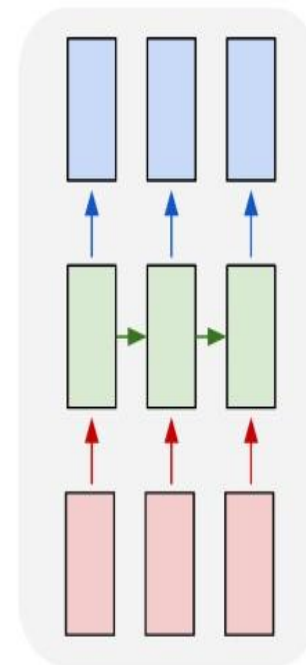
many to one



many to many

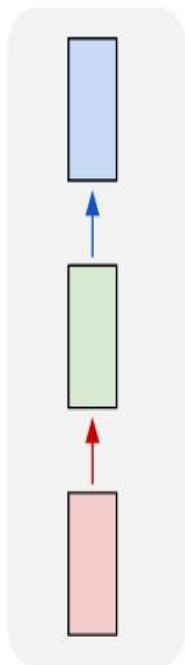


many to many

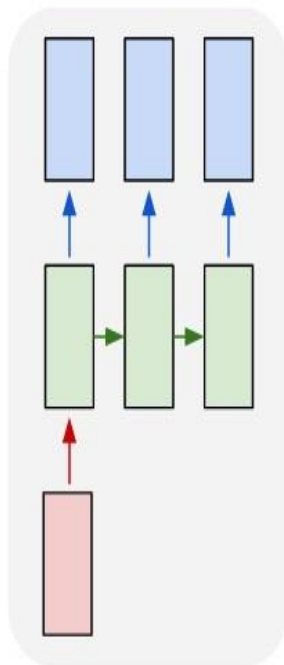


e.g. **Sentiment Classification**
sequence of words -> sentiment

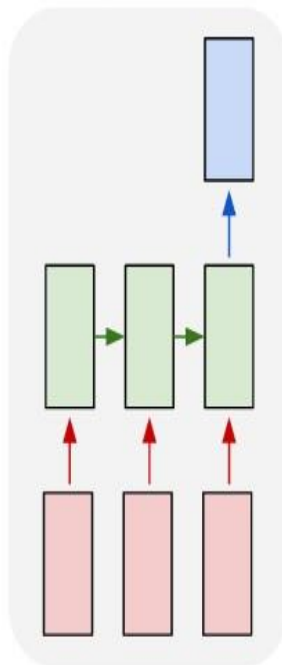
one to one



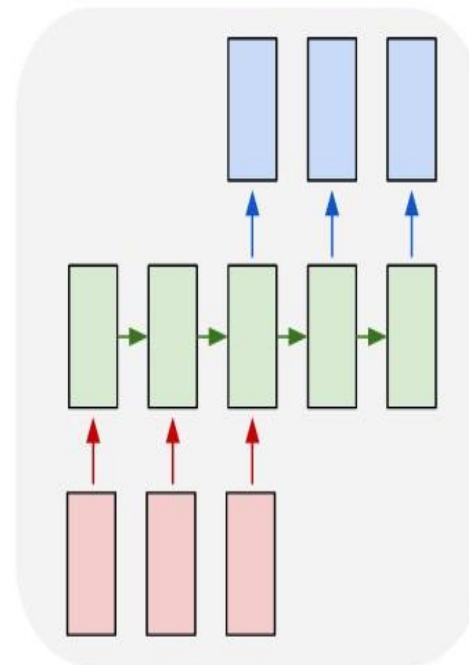
one to many



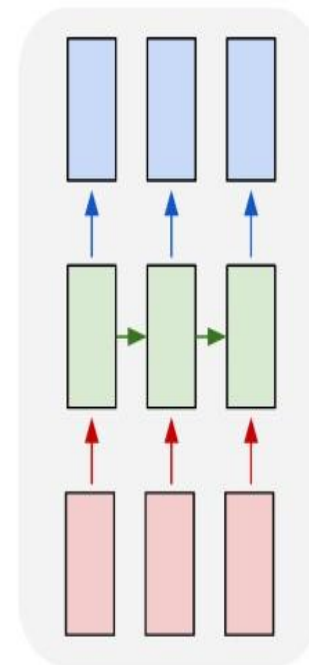
many to one



many to many

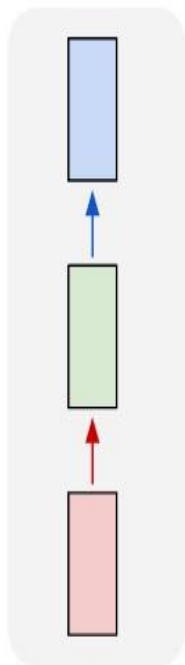


many to many

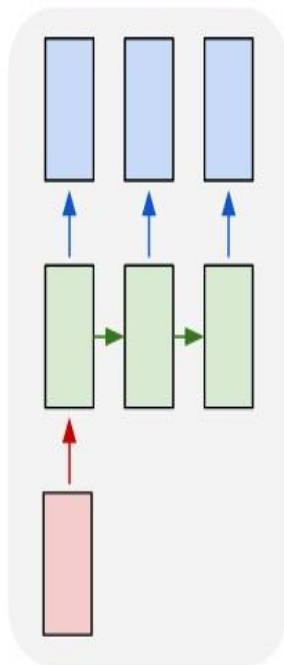


e.g. **Machine Translation**
seq of words -> seq of words

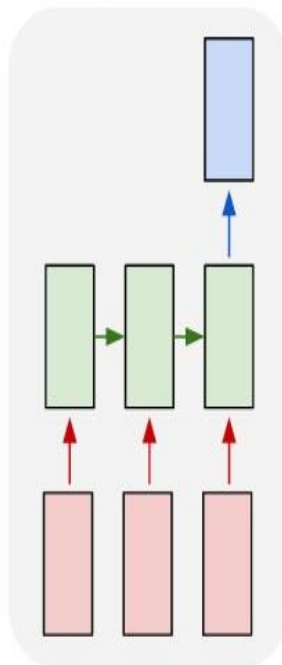
one to one



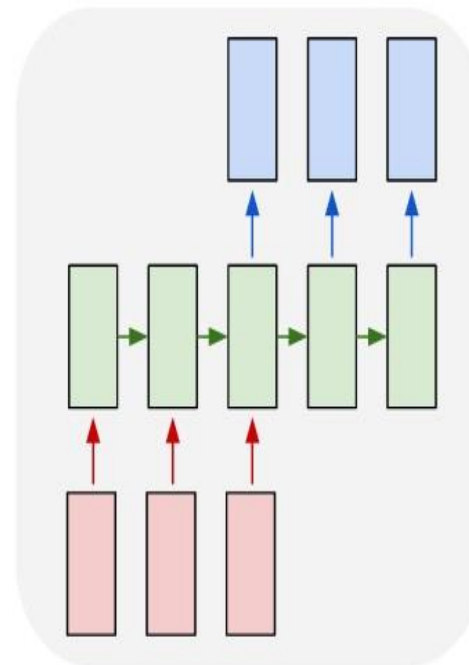
one to many



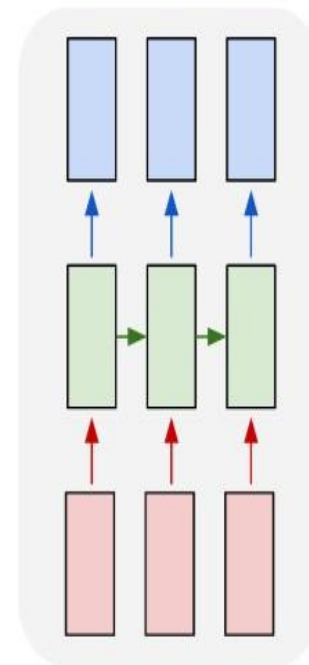
many to one



many to many



many to many



e.g. **Video classification on frame level**



A RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax} \left(U h^{(t)} + b_2 \right) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma \left(W_h h^{(t-1)} + W_e e^{(t)} + b_1 \right)$$

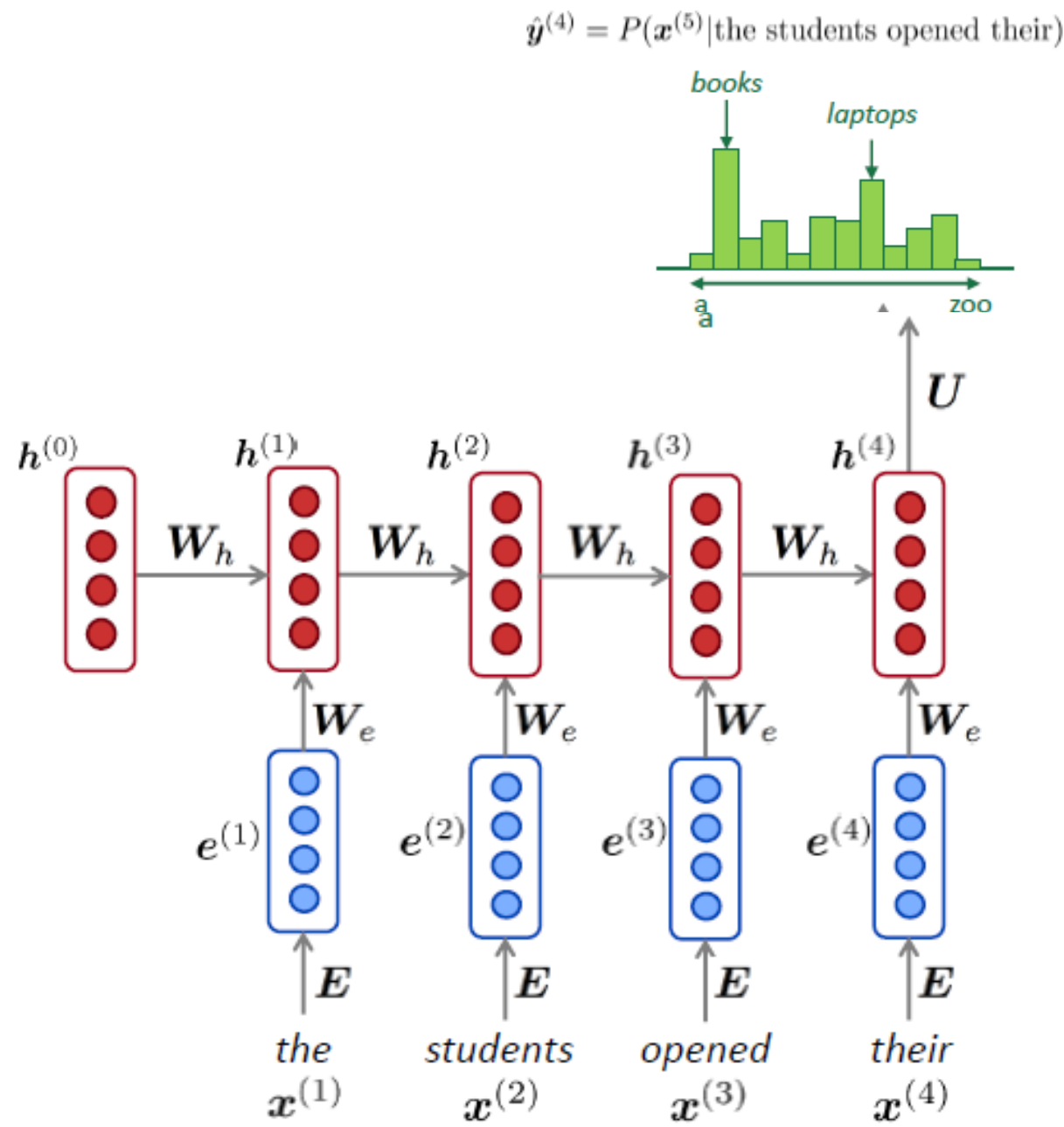
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Note: this input sequence could be much longer, but this slide doesn't have space!

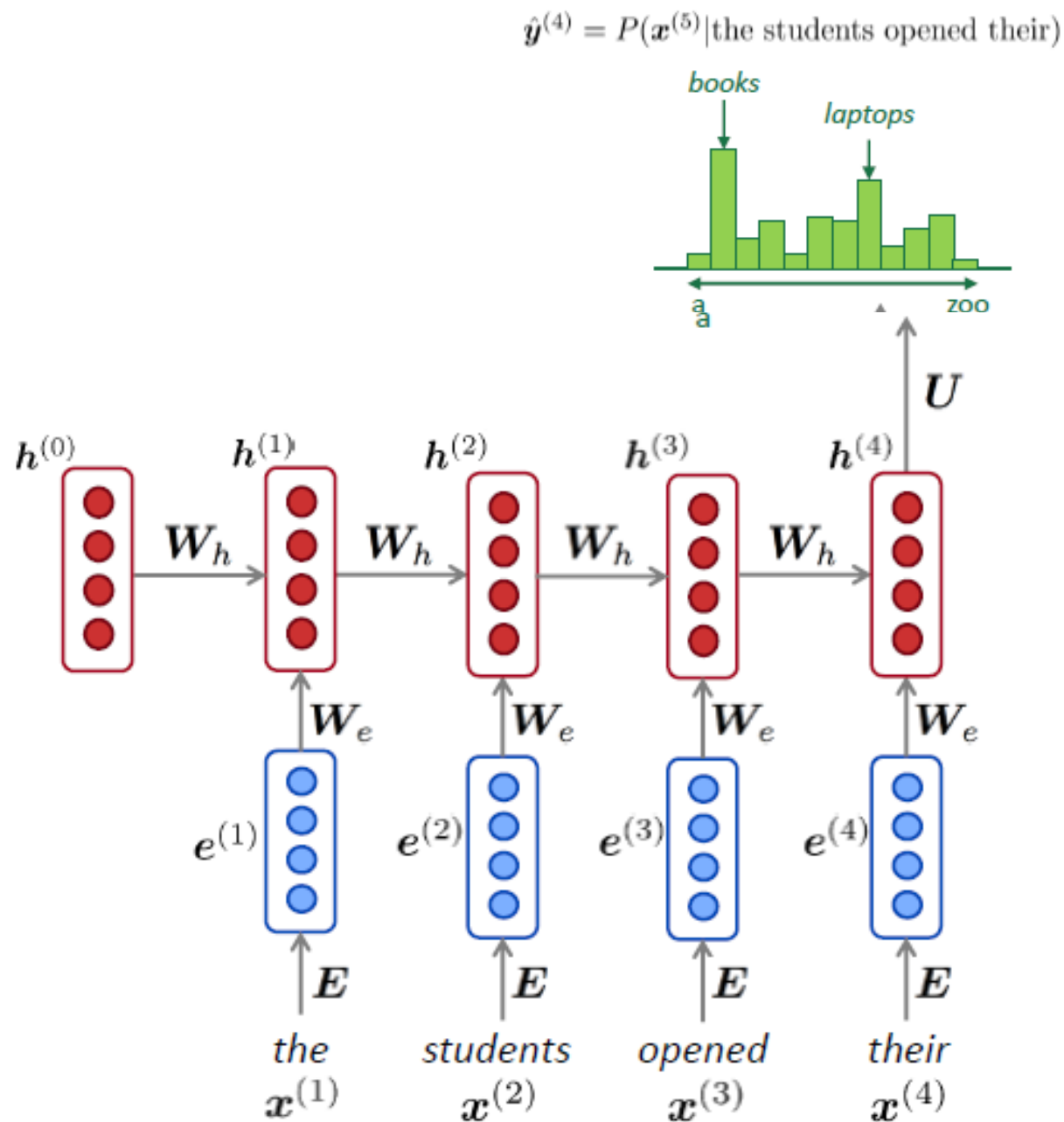
A RNN Language Model

RNN Advantages:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is **slow**
 - In practice, difficult to access information from **many steps back**
- More on these later in the course



RNN for Language Modeling (Loss Function)

Cross Entropy Loss for Language Modeling

$$\text{Output Vector} = y_w = [0 \quad 0 \quad \dots 0 \quad 1 \quad 0 \quad \dots 0]$$

$w_1 \quad w_2 \quad \dots w_{i+1} \quad w_v$

$$\hat{y}_w = [0.01 \quad 0.02 \quad 0 \quad \dots 0.3 \quad 0 \quad \dots 0.1]$$

$w_1 \quad w_2 \quad \dots w_{i+1} \quad \dots w_v$

Cross Entropy Loss

$$y_{w_1} \log \hat{y}_{w_1} + \dots y_{w_{i+1}} \log \hat{y}_{w_{i+1}} + \dots y_{w_v} \log \hat{y}_{w_v}$$

since all y_w are 0 except $y_{w_{i+1}}$ so

$$\text{Loss} = y_{w_{i+1}} \log \hat{y}_{w_{i+1}}$$

Training a RNN Language Model (Loss Function)

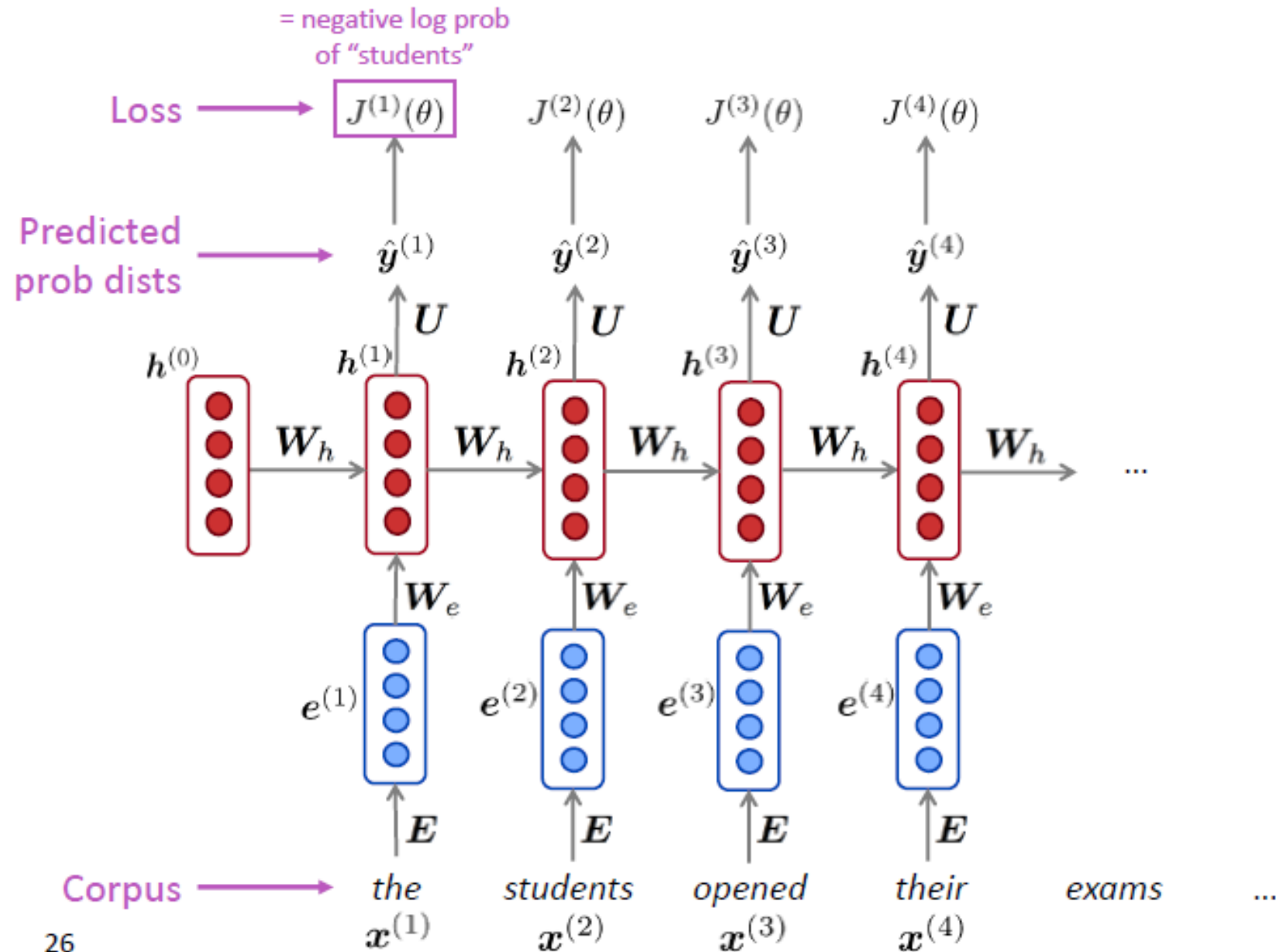
- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e. predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

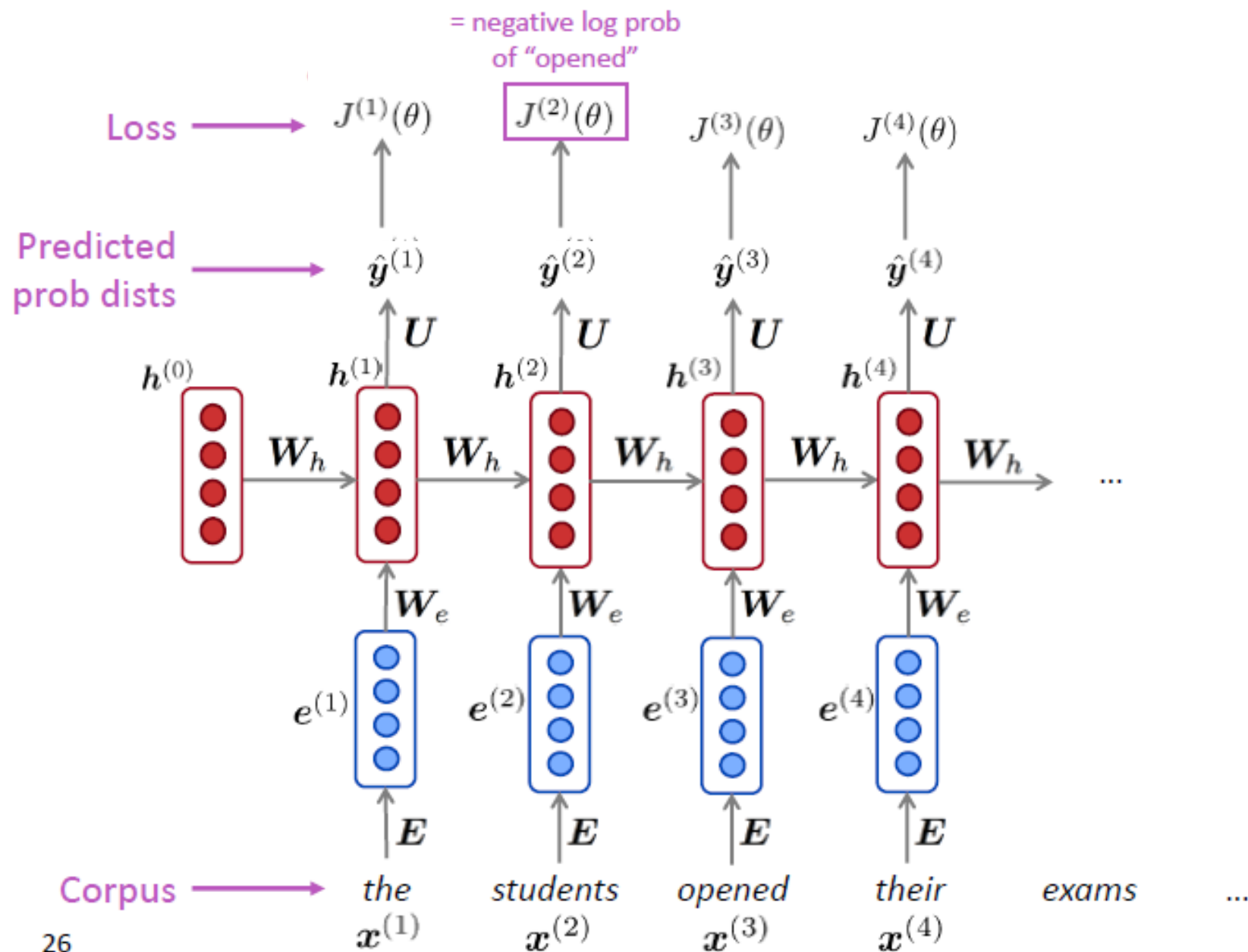
- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{x_{t+1}}^{(t)}$$

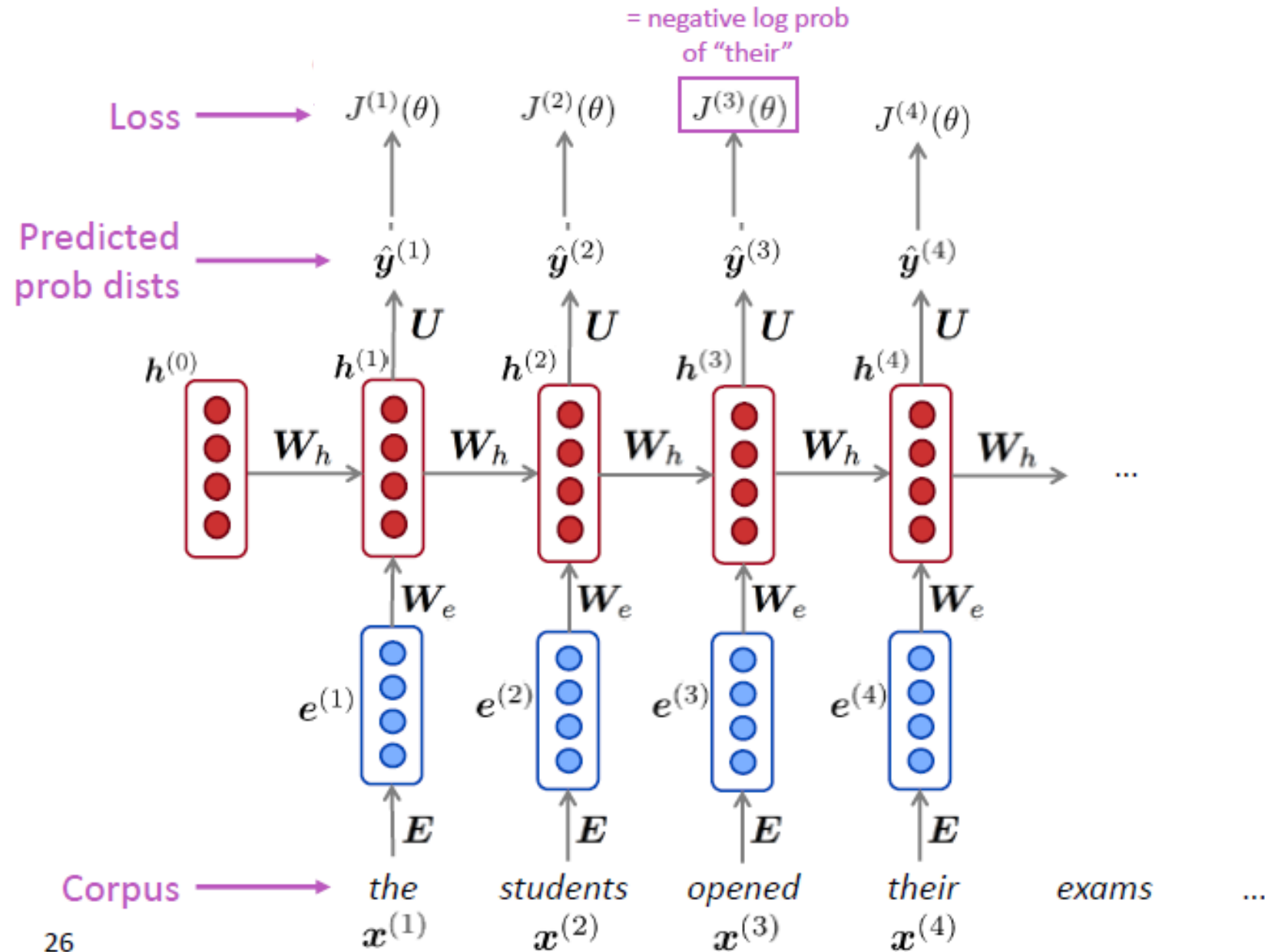
Training a RNN Language Model (Loss Function)



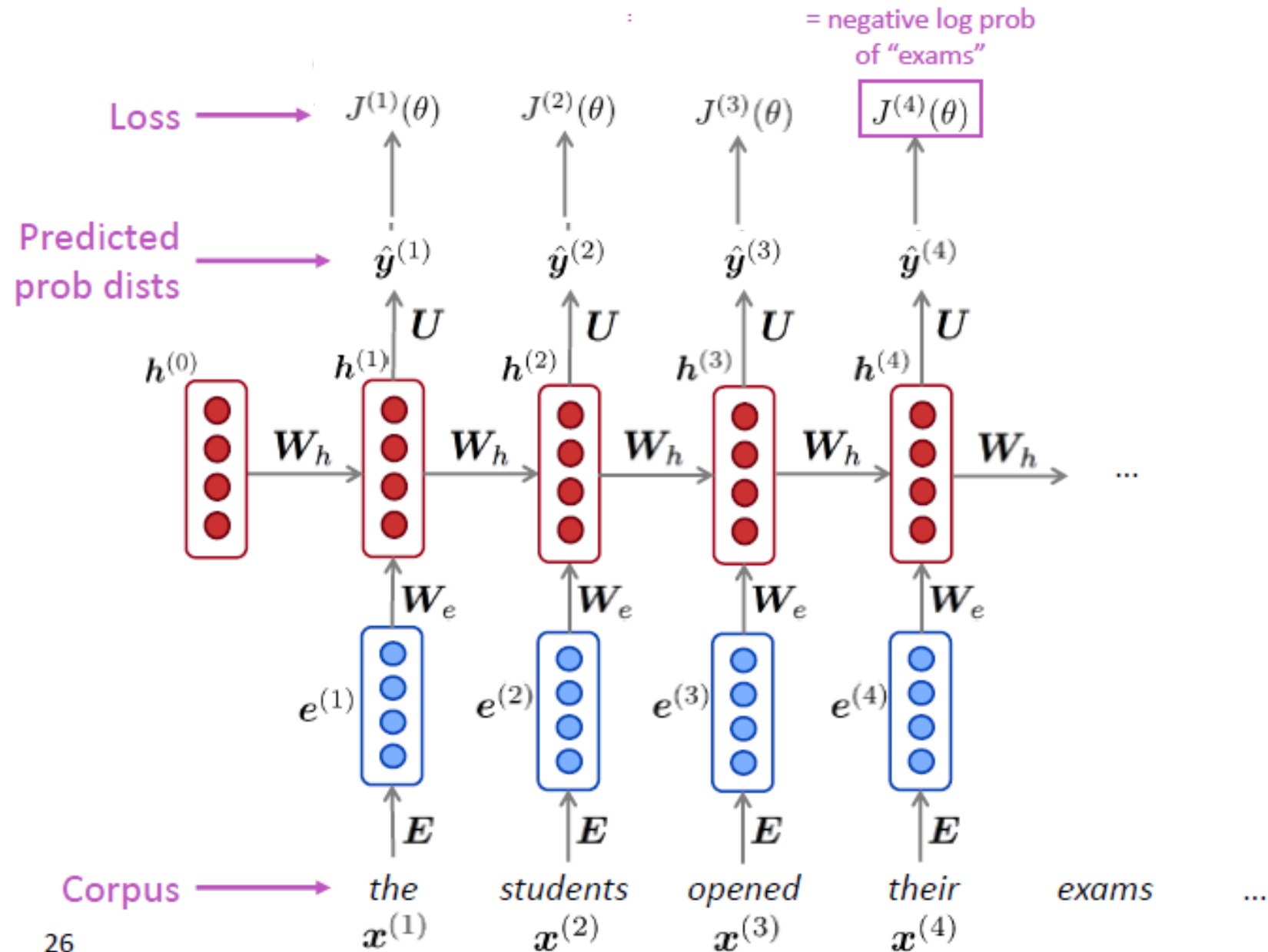
Training a RNN Language Model (Loss Function)



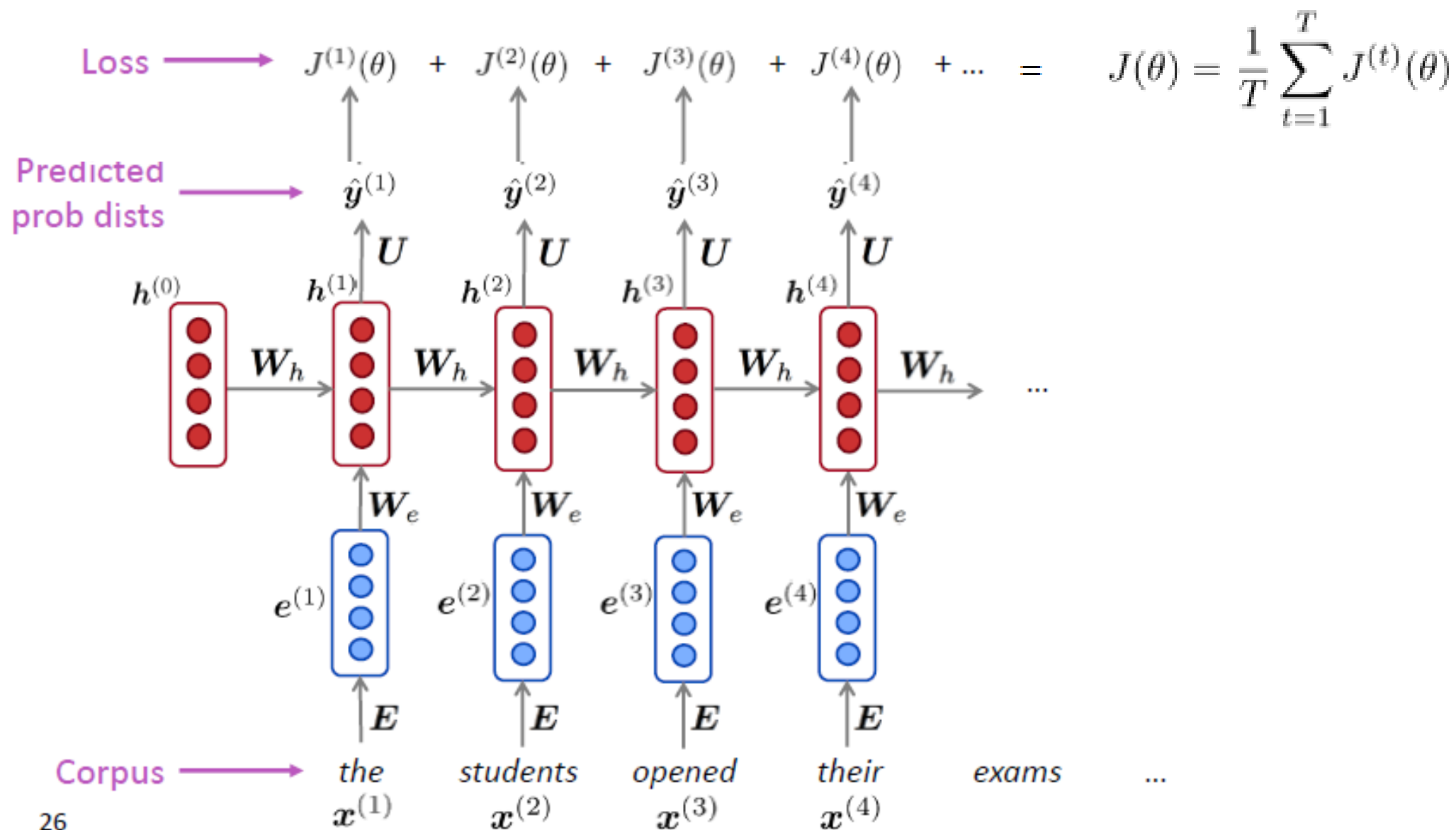
Training a RNN Language Model (Loss Function)



Training a RNN Language Model (Loss Function)



Training a RNN Language Model (Loss Function)



Training a RNN Language Model (Loss Function)

- However: Computing loss and gradients across **entire corpus** $x^{(1)}, \dots, x^{(T)}$ is **too expensive!**

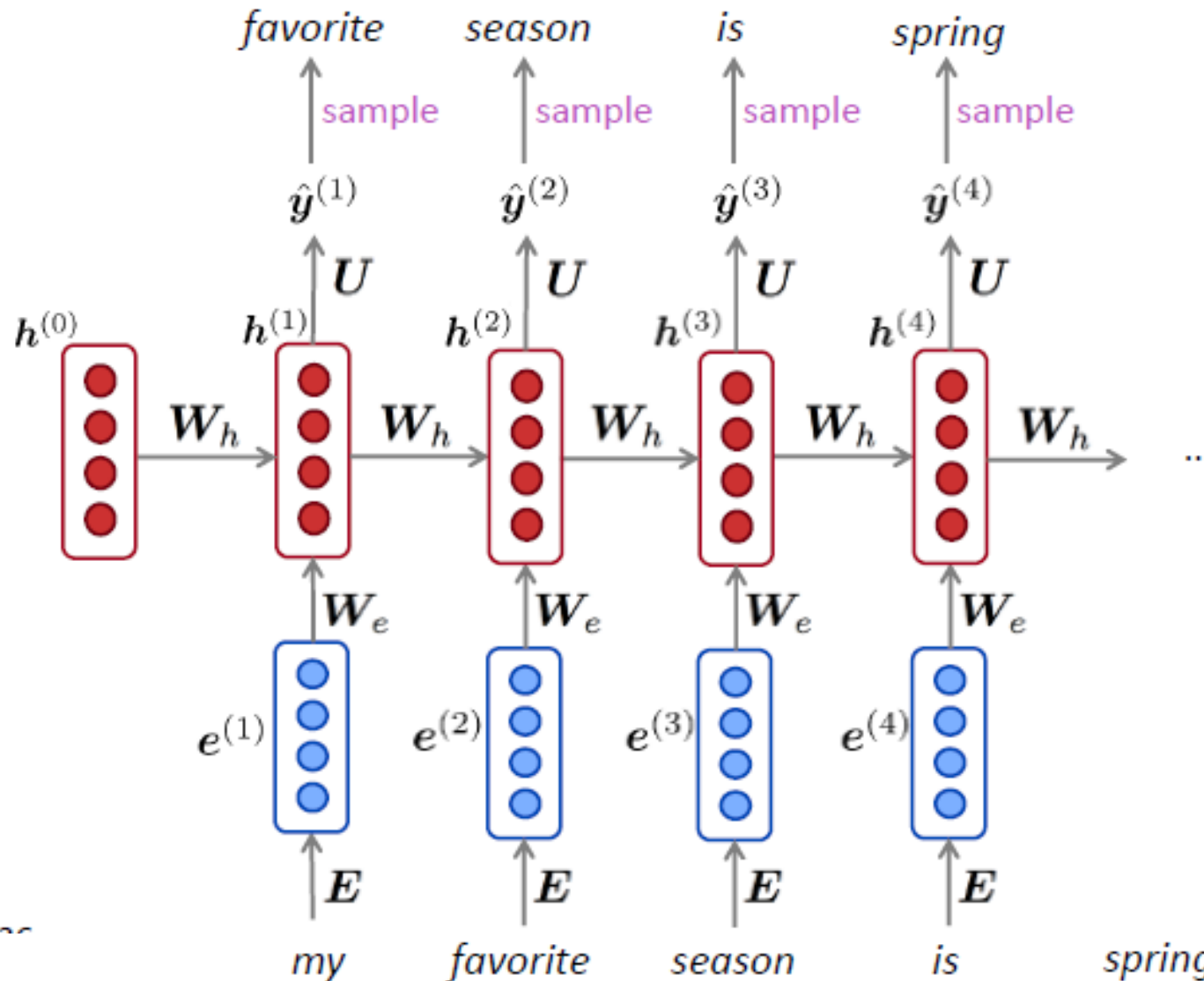
$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually a batch of sentences), compute gradients and update weights. Repeat.

Generating Text with RNN

Generating text with a RNN Language Model

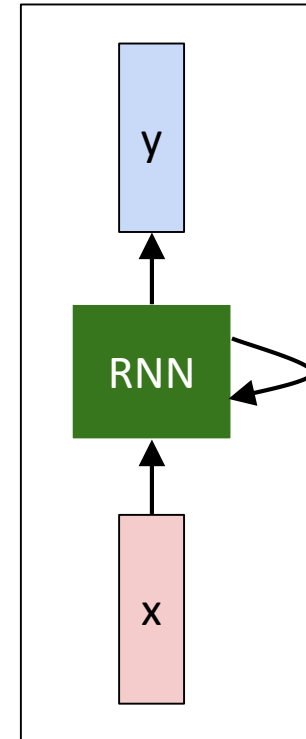
Just like a n-gram Language Model, you can use a RNN Language Model to **generate text by repeated sampling**. Sampled output is next step's input.



A Character based RNN Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

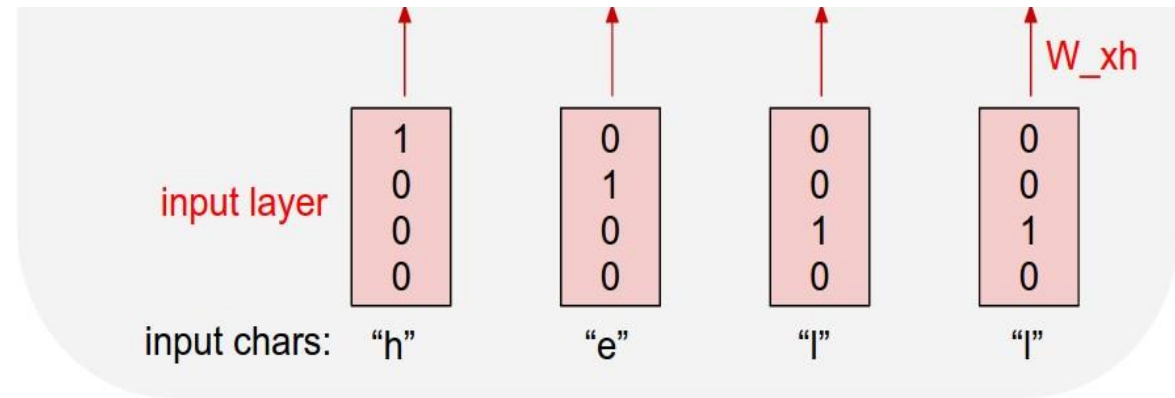


A Character based RNN Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

During training: the RNN knows the correct sequence in advance (ground truth) and uses it to guide learning.



A Character based RNN Language Model

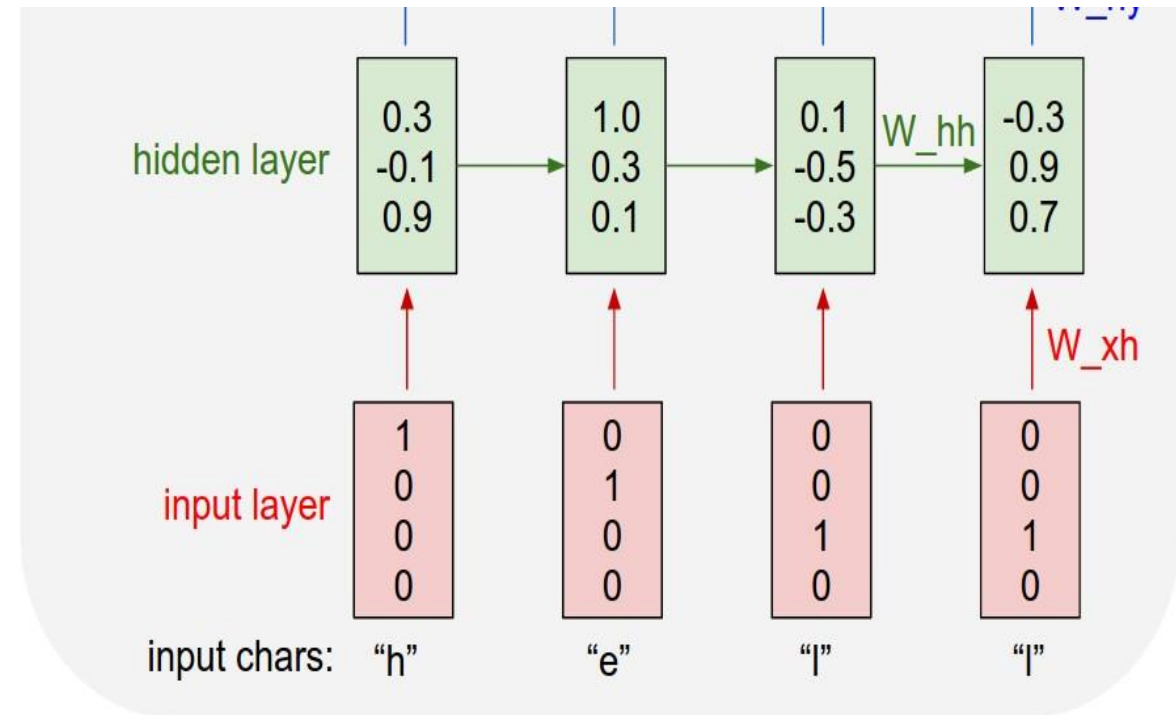
Vocabulary:

[h,e,l,o]

Example training
sequence:

“hello”

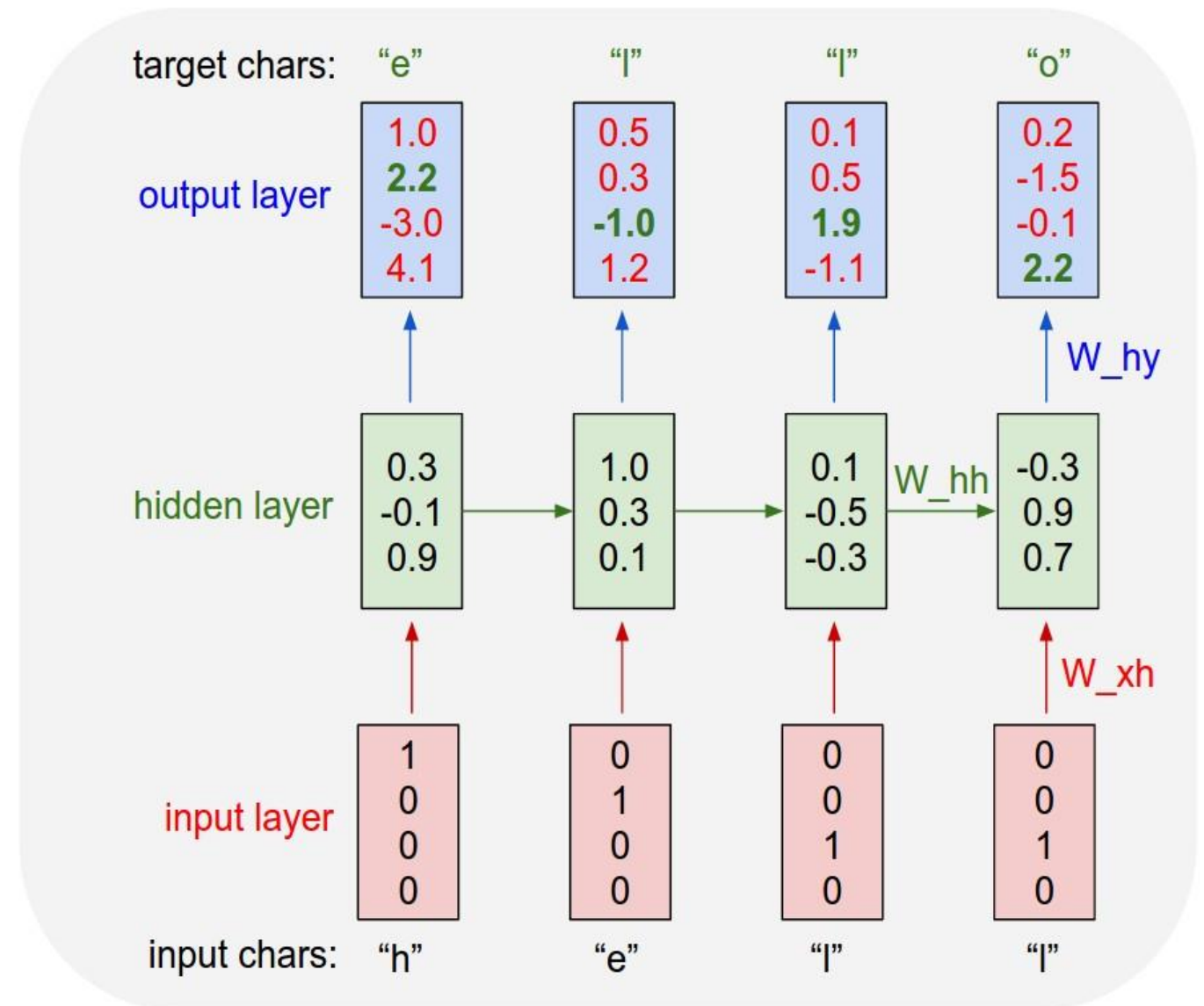
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



A Character based RNN Language Model

Vocabulary:
[h,e,l,o]

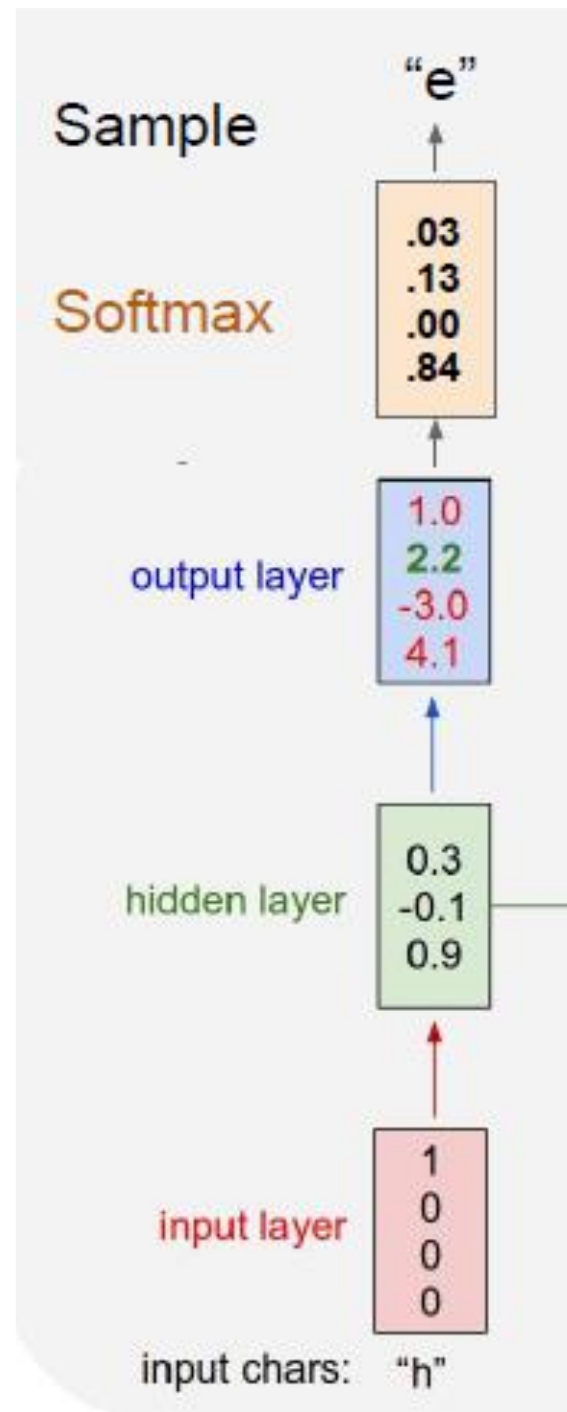
Example training
sequence:
"hello"



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model

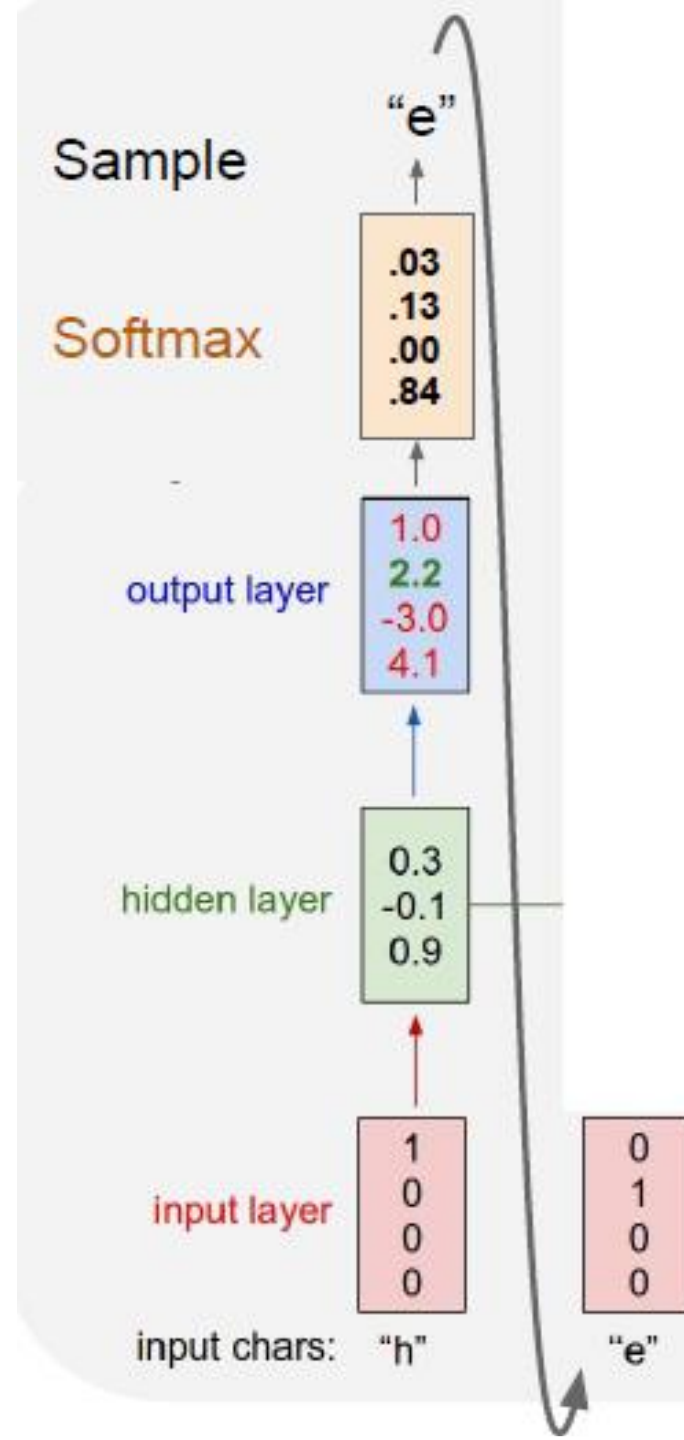


During testing (inference): the RNN doesn't have the correct answer — it must generate the sequence itself, one step at a time, using its own previous outputs as input for the next step.

Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

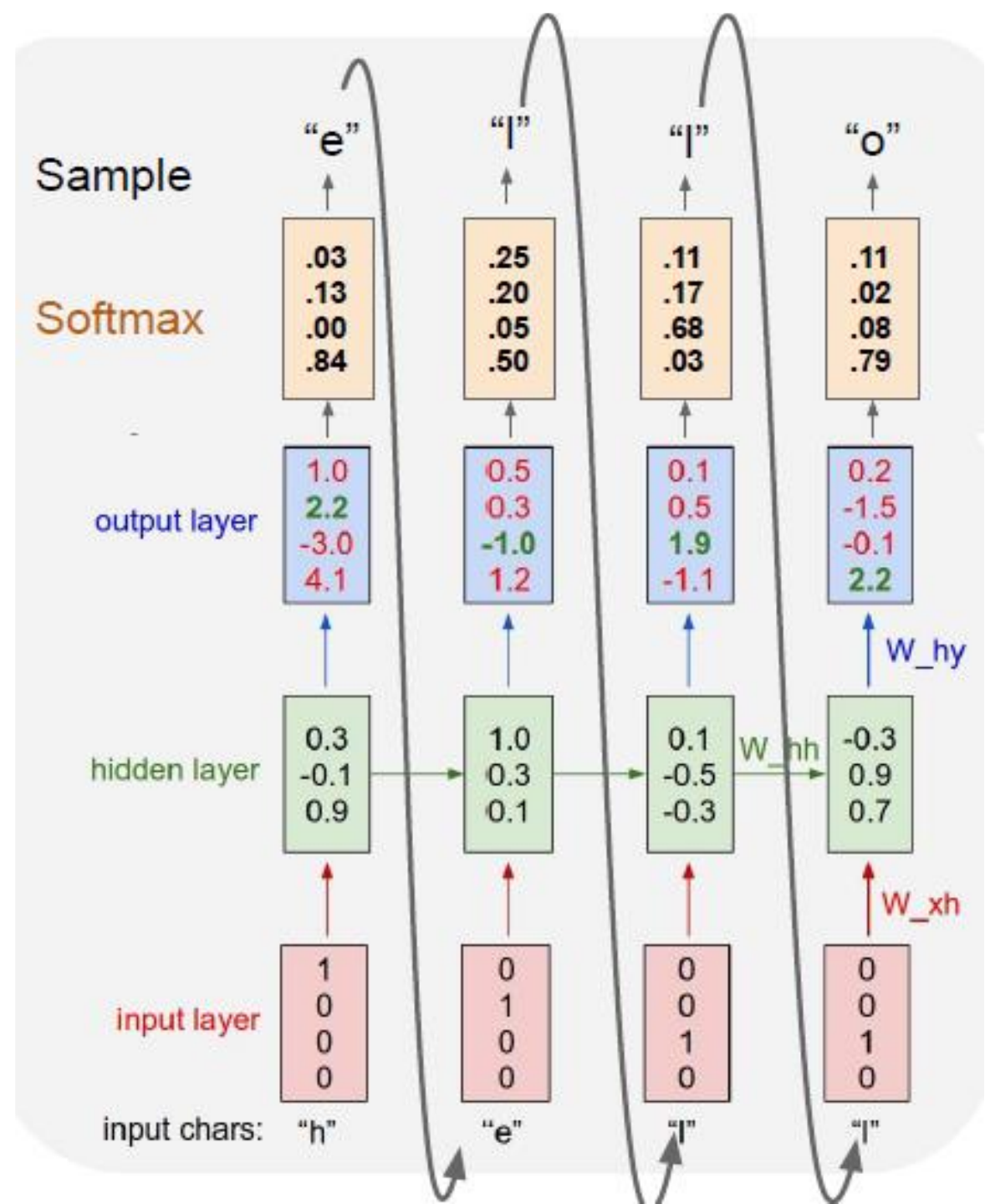
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model



A Simple RNN code based on RNN equations is available at following link

min-char-rnn.py gist: 112 lines of Python

```
1 """
2 Minimal character-level Vanilla RNN model. Written by Andrej Karpathy (@karpathy)
3 BSD License
4 """
5
6 import numpy as np
7
8 # data io
9 data = open('input.txt', 'r').read() # should be single plain text file
10 chars = list(set(data))
11 data_size, vocab_size = len(data), len(chars)
12 print 'data has %d characters, %d unique.' % (data_size, vocab_size)
13 char_to_ix = { ch:i for i, ch in enumerate(chars) }
14 ix_to_char = { ix:ch for i, ch in enumerate(chars) }
15
16 # hyperparameters
17 hidden_size = 100 # size of hidden layer of neurons
18 seq_length = 25 # number of steps to unroll the RNN for
19 learning_rate = 0.01
20
21 # model parameters
22 Wih = np.random.randn(hidden_size, vocab_size)*0.01 # input to hidden
23 Whh = np.random.randn(hidden_size, hidden_size)*0.01 # hidden to hidden
24 Why = np.random.randn(vocab_size, hidden_size)*0.01 # hidden to output
25 bhh = np.zeros([hidden_size, 1]) # hidden bias
26 by = np.zeros([vocab_size, 1]) # output bias
27
28 def lossFun(inputs, targets, hprev):
29     """
30     inputs, targets are both list of integers.
31     hprev is list array of initial hidden state
32     returns the loss, gradients on model parameters, and last hidden state
33     """
34     xs, hs, ys, ps = [], [], [], []
35     h[-1] = np.copy(hprev)
36     loss = 0
37     # forward pass
38     for t in xrange(len(inputs)):
39         xs.append([vocab_size, 1]) # encode in 2-of-k representation
40         xs[t][vocab_size] = 1
41         hs.append(np.dot(Wih, xs[t]) + np.dot(Whh, hs[t-1]) + bhh) # hidden state
42         ys.append(np.dot(Why, hs[t]) + by) # unnormalized log probabilities for next chars
43         ps.append(np.exp(ys[t]) / np.sum(np.exp(ys[t]))) # probabilities for next chars
44         loss += -np.log(ps[t][targets[t], 0]) # softmax (cross-entropy) loss
45     # backwards pass: compute gradients going backwards
46     dhs, dWih, dWhy = np.zeros_like(Wih), np.zeros_like(Whh), np.zeros_like(Why)
47     dhs, dby = np.zeros_like(bh), np.zeros_like(by)
48     dnext = np.zeros_like(hs[0])
49     for t in reversed(xrange(len(inputs))):
50         dy = np.copy(ps[t])
51         dy[targets[t]] -= 1 # backprop into y
52         dby += np.dot(dy, hs[t].T)
53         dhs += dy
54         dh = np.dot(Why.T, dy) + dnext # backprop into h
55         dhraw = (1 - hs[t] * hs[t]) * dh # backprop through tanh nonlinearity
56         dhs += dhraw
57         dWih += np.dot(dhraw, xs[t].T)
58         dWhh += np.dot(dhraw, hs[t-1].T)
59         dnext = np.dot(Wih.T, dhraw)
60     for dparam in [dWih, dWhh, dWhy, dhs, dby]:
61         np.clip(dparam, -0.1, 0.1, out=dparam) # clip to mitigate exploding gradients
62     return loss, dWih, dWhh, dWhy, dhs, dby, hs[-1]
63
64 def sample(h, seed_ix, n):
65     """
66     sample a sequence of integers from the model
67     h is memory state, seed_ix is seed letter for first time step
68     """
69     x = np.zeros([vocab_size, 1])
70     s[seed_ix] = 1
71     ixes = []
72     for t in xrange(n):
73         h = np.tanh(np.dot(Woh, x) + np.dot(Wih, h) + bh)
74         y = np.dot(Why, h) + by
75         p = np.exp(y) / np.sum(np.exp(y))
76         ix = np.random.choice(range(vocab_size), p=p.ravel())
77         x = np.zeros([vocab_size, 1])
78         s[ix] = 1
79         ixes.append(ix)
80     return ixes
81
82 n, p = 0, 0
83 Wih, Whh, Why = np.zeros_like(Wih), np.zeros_like(Whh), np.zeros_like(Why)
84 Whh, Why = np.zeros_like(bh), np.zeros_like(by) # memory variables for Adagrad
85 smooth_loss = -np.log(1.0/vocab_size)*seq_length # loss at iteration 0
86 while True:
87     # prepare inputs (we're sweeping from left to right in steps seq_length long)
88     if p+seq_length > len(data) or p == 0:
89         hprev = np.zeros([hidden_size, 1]) # reset mem memory
90         p = 0 # go from start of data
91     inputs = [char_to_ix[ch] for ch in data[p:p+seq_length]]
92     targets = [char_to_ix[ch] for ch in data[p+1:p+seq_length+1]]
93
94     # sample from the model now and then
95     if p % 100 == 0:
96         sample_ix = sample(hprev, inputs[0], 100)
97         txt = ''.join(ix_to_char[ix] for ix in sample_ix)
98         print '....%s\n' % (txt, )
99
100     # forward seq_length characters through the net and unroll gradients
101     loss, dWih, dWhh, dWhy, dhs, dby, hprev = lossFun(inputs, targets, hprev)
102     smooth_loss = smooth_loss * 0.999 + loss * 0.001
103     if p % 100 == 0: print 'iter %d, loss: %f' % (n, smooth_loss) # print progress
104
105     # perform parameter update with Adagrad
106     for dparam, dparam_new in zip([dWih, Whh, Why, bh, by],
107                                   [dWih, dWhh, dWhy, dhs, dby]):
108         dparam_new += dparam * dparam / np.sqrt(dparam + 1e-8) # adagrad update
109         dparam = dparam_new
110
111     p += seq_length # move data pointer
112     n += 1 # iteration counter
```

(<https://gist.github.com/karpathy/d4dee566867f8291f086>)

(<https://gist.github.com/karpathy/d4dee566867f8291f086>)

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on Obama speeches:



The United States will step up to the cost of a new challenges of the American people that will share the fact that we created the problem. They were attacked and so that they have to say that all the task of the final days of war that I will not be able to get this done.

Source: <https://medium.com/@samim/obama-rnn-machine-generated-political-speeches-c8abd18a2ea0>

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on *Harry Potter*:



“Sorry,” Harry shouted, panicking—“I’ll leave those brooms in London, are they?”

“No idea,” said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle Charms, from Harry’s shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn’t felt it seemed. He reached the teams too.

Source: <https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6>

Generating text with a RNN Language Model

- Let's have some fun!
- You can train a RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on **recipes**:

Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies, Cookies
Yield: 6 Servings

2 tb Parmesan cheese — chopped
1 c Coconut milk
3 Eggs, beaten

Place each pasta over layers of lumps. Shape mixture into the moderate oven and simmer until firm. Serve hot in bodied fresh, mustard, orange and cheese.

Combine the cheese and salt together the dough in a large skillet; add the ingredients and stir in the chocolate and pepper.



Source: <https://gist.github.com/nylki/1efbaa36635956d35bcc>

at first:

tyntd-iafhatawiaoihrdemot lytdws e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrqd t o idoe ns,smtt h ne etie h,hregtrs nigtike,aoaenns lng

train more

"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwv fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."

train more

Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.

train more

"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftended him.
Pierre aking his soul came to the packs and drove up his father-in-law women.

RNN Trained on
some novel

The Stacks Project

[home](#) [about](#) [tags explained](#) [tag lookup](#) [browse](#) [search](#) [bibliography](#) [recent comments](#) [blog](#) [add slogans](#)

Browse chapters

Part	Chapter	online	TeX source	view pdf
Preliminaries				
	1. Introduction	online	tex 	pdf 
	2. Conventions	online	tex 	pdf 
	3. Set Theory	online	tex 	pdf 
	4. Categories	online	tex 	pdf 
	5. Topology	online	tex 	pdf 
	6. Sheaves on Spaces	online	tex 	pdf 
	7. Sites and Sheaves	online	tex 	pdf 
	8. Stacks	online	tex 	pdf 
	9. Fields	online	tex 	pdf 
	10. Commutative Algebra	online	tex 	pdf 

Parts

1. [Preliminaries](#)
2. [Schemes](#)
3. [Topics in Scheme Theory](#)
4. [Algebraic Spaces](#)
5. [Topics in Geometry](#)
6. [Deformation Theory](#)
7. [Algebraic Stacks](#)
8. [Miscellany](#)

Statistics

The Stacks project now consists of

- 455910 lines of code
- 14221 tags (56 inactive tags)
- 2366 sections

Latex source

Algebra Equations and proofs using RNN

For $\bigoplus_{n=1,\dots,m}$ where $\mathcal{L}_{m,\bullet} = 0$, hence we can find a closed subset $\mathcal{H}$ in $\mathcal{H}$ and any sets $\mathcal{F}$ on X , U is a closed immersion of S , then $U \rightarrow T$ is a separated algebraic space.

Proof. Proof of (1). It also start we get

$$S = \text{Spec}(R) = U \times_X U \times_X U$$

and the comparicoly in the fibre product covering we have to prove the lemma generated by $\coprod Z \times_U U \rightarrow V$. Consider the maps M along the set of points Sch_{fppf} and $U \rightarrow U$ is the fibre category of S in U in Section, ?? and the fact that any U affine, see Morphisms, Lemma ?? . Hence we obtain a scheme S and any open subset $W \subset U$ in $Sh(G)$ such that $\text{Spec}(R') \rightarrow S$ is smooth or an

$$U = \bigcup U_i \times_{S_i} U_i$$

which has a nonzero morphism we may assume that f_i is of finite presentation over S . We claim that $\mathcal{O}_{X,x}$ is a scheme where $x, x', s'' \in S'$ such that $\mathcal{O}_{X,x'} \rightarrow \mathcal{O}'_{X',x'}$ is separated. By Algebra, Lemma ?? we can define a map of complexes $\text{GL}_{S'}(x'/S'')$ and we win. $\square$

To prove study we see that $\mathcal{F}|_U$ is a covering of $\mathcal{X}'$, and $\mathcal{T}_i$ is an object of $\mathcal{F}_{X/S}$ for $i > 0$ and $\mathcal{F}_p$ exists and let $\mathcal{F}_i$ be a presheaf of $\mathcal{O}_X$ -modules on $\mathcal{C}$ as a $\mathcal{F}$ -module. In particular $\mathcal{F} = U/\mathcal{F}$ we have to show that

$$\widetilde{M}^\bullet = \mathcal{I}^\bullet \otimes_{\text{Spec}(k)} \mathcal{O}_{S,s} - i_X^{-1} \mathcal{F}$$

is a unique morphism of algebraic stacks. Note that

$$\text{Arrows} = (Sch/S)_{fppf}^{opp}, (Sch/S)_{fppf}$$

and

$$V = \Gamma(S, \mathcal{O}) \mapsto (U, \text{Spec}(A))$$

is an open subset of X . Thus U is affine. This is a continuous map of X is the inverse, the groupoid scheme S .

Proof. See discussion of sheaves of sets. $\square$

The result for prove any open covering follows from the less of Example ?? . It may replace S by $X_{spaces, \acute{e}tale}$ which gives an open subspace of X and T equal to S_{Zar} , see Descent, Lemma ?? . Namely, by Lemma ?? we see that R is geometrically regular over S .

Lemma 0.1. Assume (3) and (3) by the construction in the description.

Suppose $X = \lim |X|$ (by the formal open covering X and a single map $\text{Proj}_X(\mathcal{A}) = \text{Spec}(B)$ over U compatible with the complex

$$\text{Set}(\mathcal{A}) = \Gamma(X, \mathcal{O}_{X, \mathcal{O}_X}).$$

When in this case of to show that $\mathcal{Q} \rightarrow \mathcal{C}_{Z/X}$ is stable under the following result in the second conditions of (1), and (3). This finishes the proof. By Definition ?? (without element is when the closed subschemes are catenary. If T is surjective we may assume that T is connected with residue fields of S . Moreover there exists a closed subspace $Z \subset X$ of X where U in X' is proper (some defining as a closed subset of the uniqueness it suffices to check the fact that the following theorem

(1) f is locally of finite type. Since $S = \text{Spec}(R)$ and $Y = \text{Spec}(R)$.

Proof. This is form all sheaves of sheaves on X . But given a scheme U and a surjective étale morphism $U \rightarrow X$. Let $U \cap U = \coprod_{i=1,\dots,n} U_i$ be the scheme X over S at the schemes $X_i \rightarrow X$ and $U = \lim_i X_i$. $\square$

The following lemma surjective restrocomposes of this implies that $\mathcal{F}_{x_0} = \mathcal{F}_{x_0} = \mathcal{F}_{X,\dots,0}$.

Lemma 0.2. Let X be a locally Noetherian scheme over S , $E = \mathcal{F}_{X/S}$. Set $\mathcal{I} = \mathcal{I}_1 \subset \mathcal{I}'_n$. Since $\mathcal{I}^n \subset \mathcal{I}^n$ are nonzero over $i_0 \leq \mathfrak{p}$ is a subset of $\mathcal{I}_{n,0} \circ \overline{A}_2$ works.

Lemma 0.3. In Situation ?? . Hence we may assume $\mathfrak{q}' = 0$.

Proof. We will use the property we see that $\mathfrak{p}$ is the next functor (??). On the other hand, by Lemma ?? we see that

$$D(\mathcal{O}_{X'}) = \mathcal{O}_X(D)$$

where K is an F -algebra where δ_{n+1} is a scheme over S . $\square$

Algebra Equations and proofs using RNN

Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$ -modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

.

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\text{étale}}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \rightarrow \mathcal{F}$ of $\mathcal{O}$ -modules. □

Lemma 0.2. *This is an integer Z is injective.*

Proof. See Spaces, Lemma ?? □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \rightarrow Y' \rightarrow Y \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

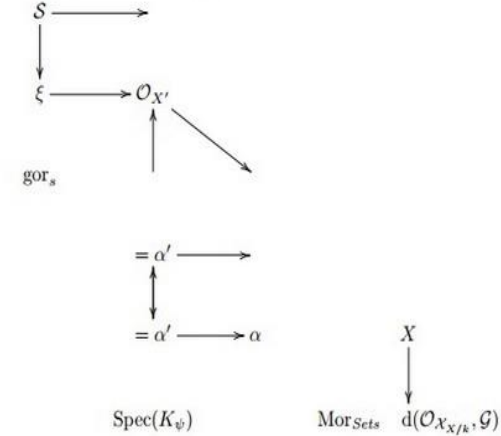
be a morphism of algebraic spaces over S and Y .

Proof. Let X be a nonzero scheme of X . Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$ -modules. The following are equivalent

- (1) $\mathcal{F}$ is an algebraic space over S .
- (2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_* . This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \text{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U . □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.

A reduced above we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_{\overline{x}} \rightarrow \mathcal{O}_{X_{\text{étale}}}^{-1} \longrightarrow \mathcal{O}_{X_{\lambda}}^{-1} \mathcal{O}_{X_{\lambda}}(\mathcal{O}_{X_{\eta}}^{\vee})$$

is an isomorphism of covering of $\mathcal{O}_{X_{\lambda}}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.

The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$ -algebra with $\mathcal{F}$ are opens of finite type over S .

If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_{\lambda}}$ is a closed immersion, see Lemma ?? . This is a sequence of $\mathcal{F}$ is a similar morphism.

C Code generation using RNN

This screenshot shows the GitHub repository for the Linux kernel source tree, maintained by Linus Torvalds. The repository is highly active, with over 520,000 commits and 5,039 contributors. The main branch is 'master', and the latest commit is 4b1786927d. The repository is organized into a directory tree with various subdirectories like Documentation, arch, block, crypto, drivers, firmware, fs, include, init, and io. The right sidebar provides options to view the code, pull requests (74), pulse, graphs, and clone the repository using HTTPS, SSH, or Subversion. There are also buttons for cloning to the desktop and downloading the ZIP file.

torvalds / linux

Linux kernel source tree

520,037 commits 1 branch 420 releases 5,039 contributors

branch: master - linux / +

Merge branch 'drm-fixes' of git://people.freedesktop.org/~airlied/linux

torvalds authored 9 hours ago latest commit 4b1786927d

Documentation	Merge git://git.kernel.org/pub/scm/linux/kernel/git/hab/target-pending	6 days ago
arch	Merge branch 'x86-urgent-for-linus' of git://git.kernel.org/pub/scm/l...	a day ago
block	block: discard bdi_unregister() in favour of bdi_destroy()	9 days ago
crypto	Merge git://git.kernel.org/pub/scm/linux/kernel/git/herbert/crypto-2.6	10 days ago
drivers	Merge branch 'drm-fixes' of git://people.freedesktop.org/~airlied/linux	9 hours ago
firmware	firmware/ihex2fw.c: restore missing default in switch statement	2 months ago
fs	vfs: read file_handle only once in handle_to_path	4 days ago
include	Merge branch 'perf-urgent-for-linus' of git://git.kernel.org/pub/scm/...	a day ago
init	init: fix regression by supporting devices with major:minor:offset fo...	a month ago
io	Merge branch 'for-linus' of git://git.kernel.org/pub/scm/linux/kernel...	a month ago

Code

Pull requests 74

Pulse

Graphs

HTTPS clone URL

https://github.c

You can clone with [HTTPS](#), [SSH](#), or [Subversion](#).

Clone in Desktop

Download ZIP

```

static void do_command(struct seq_file *m, void *v)
{
    int column = 32 << (cmd[2] & 0x80);
    if (state)
        cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
    else
        seq = 1;
    for (i = 0; i < 16; i++) {
        if (k & (1 << i))
            pipe = (in_use & UMXTHREAD_UNCCA) +
                ((count & 0x00000000ffffffff) & 0x000000f) << 8;
        if (count == 0)
            sub(pid, ppc_md.kexec_handle, 0x20000000);
        pipe_set_bytes(i, 0);
    }
    /* Free our user pages pointer to place camera if all dash */
    subsystem_info = &of_changes[PAGE_SIZE];
    rek_controls(offset, idx, &soffset);
    /* Now we want to deliberately put it to device */
    control_check_polarity(&context, val, 0);
    for (i = 0; i < COUNTER; i++)
        seq_puts(s, "policy ");
}

```

Generated C code

```
/*
 * Copyright (c) 2006-2010, Intel Mobile Communications. All rights reserved.
 *
 * This program is free software; you can redistribute it and/or modify it
 * under the terms of the GNU General Public License version 2 as published by
 * the Free Software Foundation.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 *
 * GNU General Public License for more details.
 *
 * You should have received a copy of the GNU General Public License
 * along with this program; if not, write to the Free Software Foundation,
 * Inc., 675 Mass Ave, Cambridge, MA 02139, USA.
 */

#include <linux/kexec.h>
#include <linux/errno.h>
#include <linux/io.h>
#include <linux/platform_device.h>
#include <linux/multi.h>
#include <linux/ckevent.h>

#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>
```

Generated C code

Generated C code

```
#include <asm/io.h>
#include <asm/prom.h>
#include <asm/e820.h>
#include <asm/system_info.h>
#include <asm/setew.h>
#include <asm/pgproto.h>

#define REG_PG    vesa_slot_addr_pack
#define PFM_NOCOMP AFSR(0, load)
#define STACK_DDR(type)      (func)

#define SWAP_ALLOCATE(nr)      (e)
#define emulate_sigs()  arch_get_unaligned_child()
#define access_rw(TST)  asm volatile("movd %!esp, %0, %3" : : "r" (0)); \
    if (__type & DO_READ)

static void stat_PC_SEC __read_mostly offsetof(struct seq_argsqueue, \
    pC>[1]);

static void
os_prefix(unsigned long sys)
{
#ifdef CONFIG_PREEMPT
    PUT_PARAM_RAID(2, sel) = get_state_state();
    set_pid_sum((unsigned long)state, current_state_str(),
        (unsigned long)-1->lr_full; low;
}
}
```

RNNs have greatly improved perplexity

Model	Perplexity
Interpolated Kneser-Ney 5-gram (Chelba et al., 2013)	67.6
RNN-1024 + MaxEnt 9-gram (Chelba et al., 2013)	51.3
RNN-2048 + BlackOut sampling (Ji et al., 2015)	68.3
Sparse Non-negative Matrix factorization (Shazeer et al., 2015)	52.9
LSTM-2048 (Jozefowicz et al., 2016)	43.7
2-layer LSTM-8192 (Jozefowicz et al., 2016)	30
Ours small (LSTM-2048)	43.9
Ours large (2-layer LSTM-2048)	39.8

n-gram model →

Increasingly complex RNNs ↓

Perplexity improves (lower is better)

Why should we care about Language Modeling?

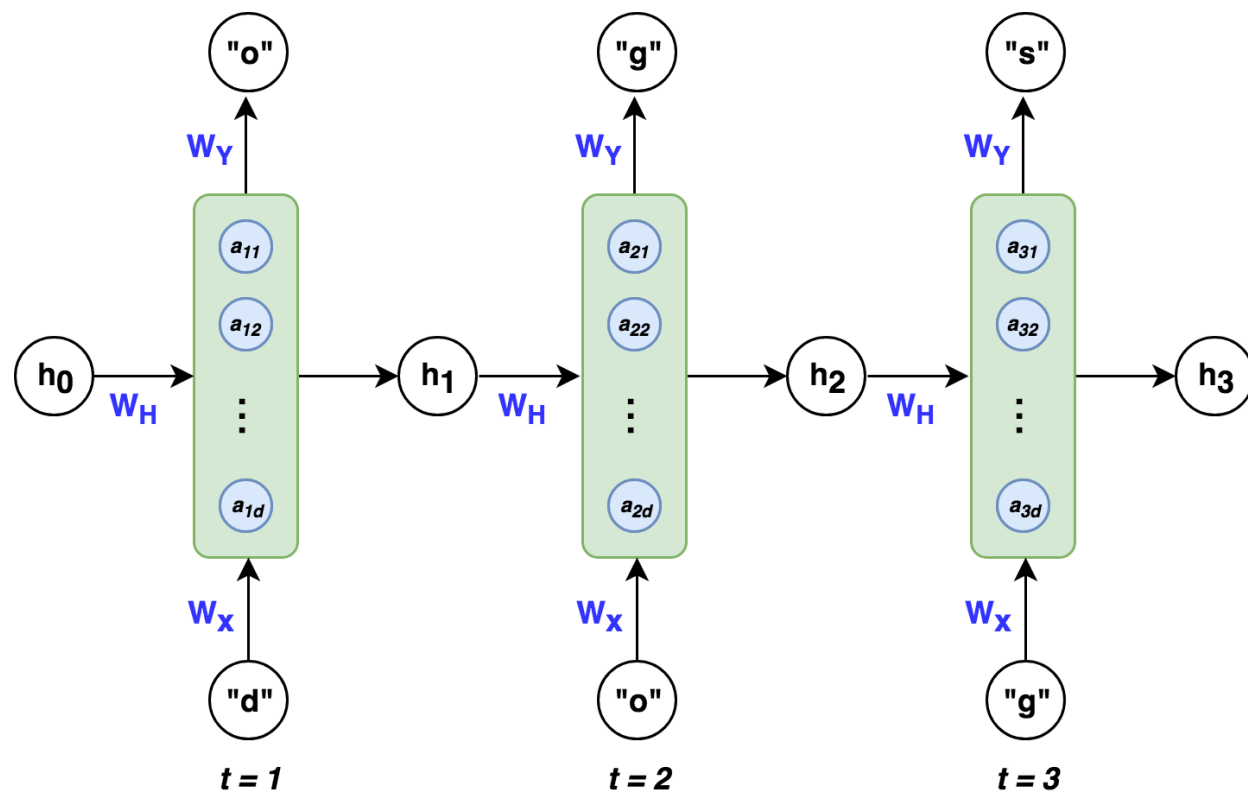
- Language Modeling is a **benchmark task** that helps us **measure our progress** on understanding language
- Language Modeling is a **subcomponent** of many NLP tasks, especially those involving **generating text** or **estimating the probability of text**:
 - Predictive typing
 - Speech recognition
 - Handwriting recognition
 - Spelling/grammar correction
 - Authorship identification
 - Machine translation
 - Summarization
 - Dialogue
 - etc.

Task 1

- Draw RNN architecture diagram and write equations along with dimensions of all layers and weight matrices for following language model:
- input sequence of length 7 (lets say 7 words).
- Hidden layer units are 5.
- $V = \text{vocabulary} = 12$

RNN Back Propagation

RNN Forward Pass

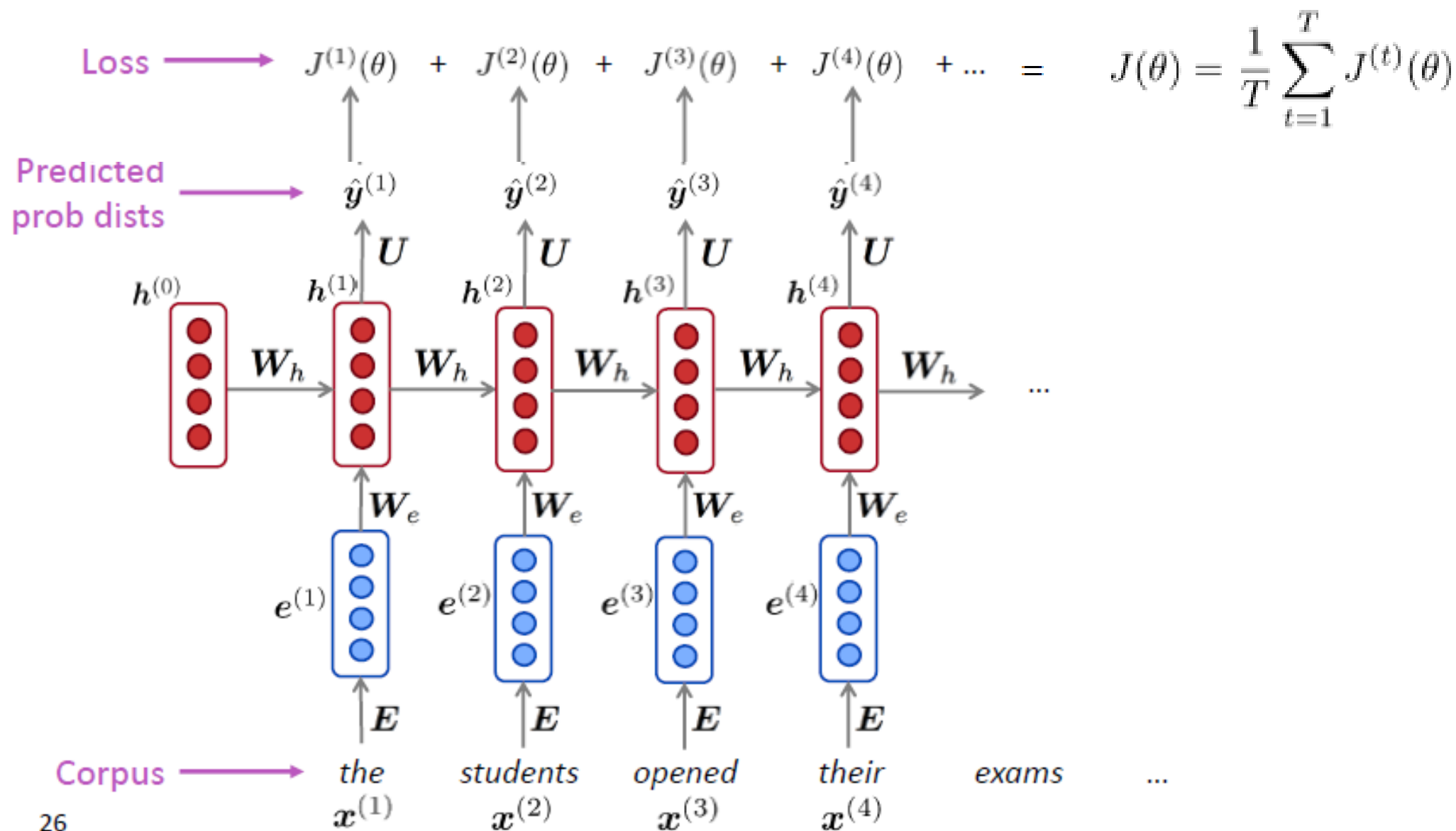


$$a_t = W_H h_{t-1} + W_X X_t \quad \text{Hidden Nodes}$$

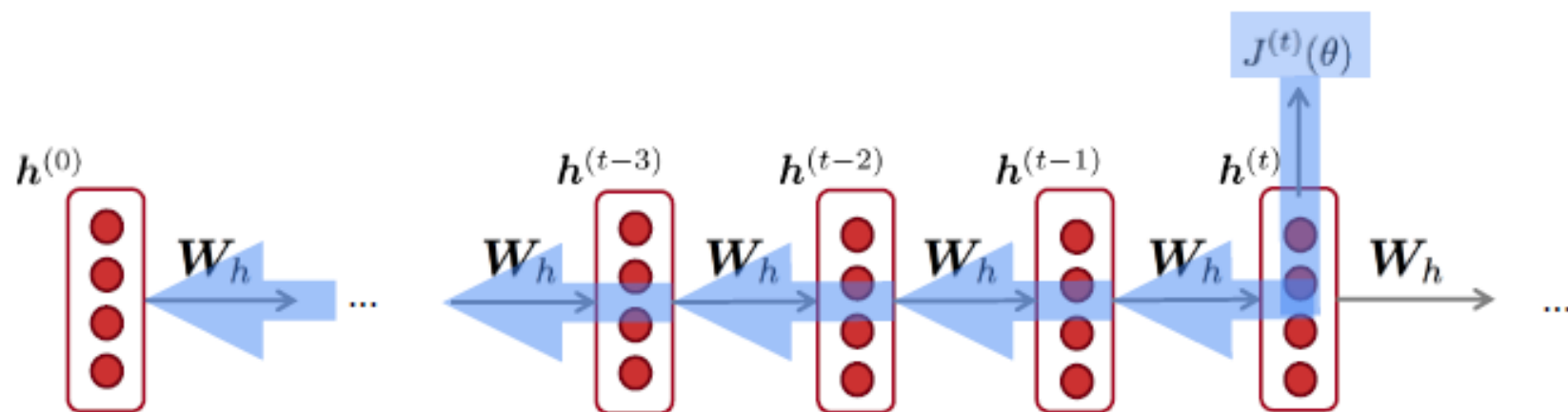
Output From Hidden State $\rightarrow h_t = \underbrace{\tanh(a_t)}_{\text{Hidden State}}$ Activation Function

Prediction at time $t \rightarrow y_t = \text{softmax}(W_Y h_t)$

Training a RNN Language Model (Loss Function)



Backpropagation for RNNs



$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$$

Question: How do we calculate this?

Answer: Backpropagate over timesteps $i=t, \dots, 0$, summing gradients as you go. This algorithm is called “backpropagation through time”

Backpropagation for RNNs

- Given a multivariable function $f(x, y)$, and two single variable functions $x(t)$ and $y(t)$, here's what the multivariable chain rule says:

$$\underbrace{\frac{d}{dt} f(x(t), y(t))}_{\text{Derivative of composition function}} = \frac{\partial f}{\partial x} \frac{dx}{dt} + \frac{\partial f}{\partial y} \frac{dy}{dt}$$

Derivative of composition function

RNN can model long term dependency

The brown and black dog, which was playing with the cat, was a german shepherd.

(x₂)

(x₄)

(x₅)

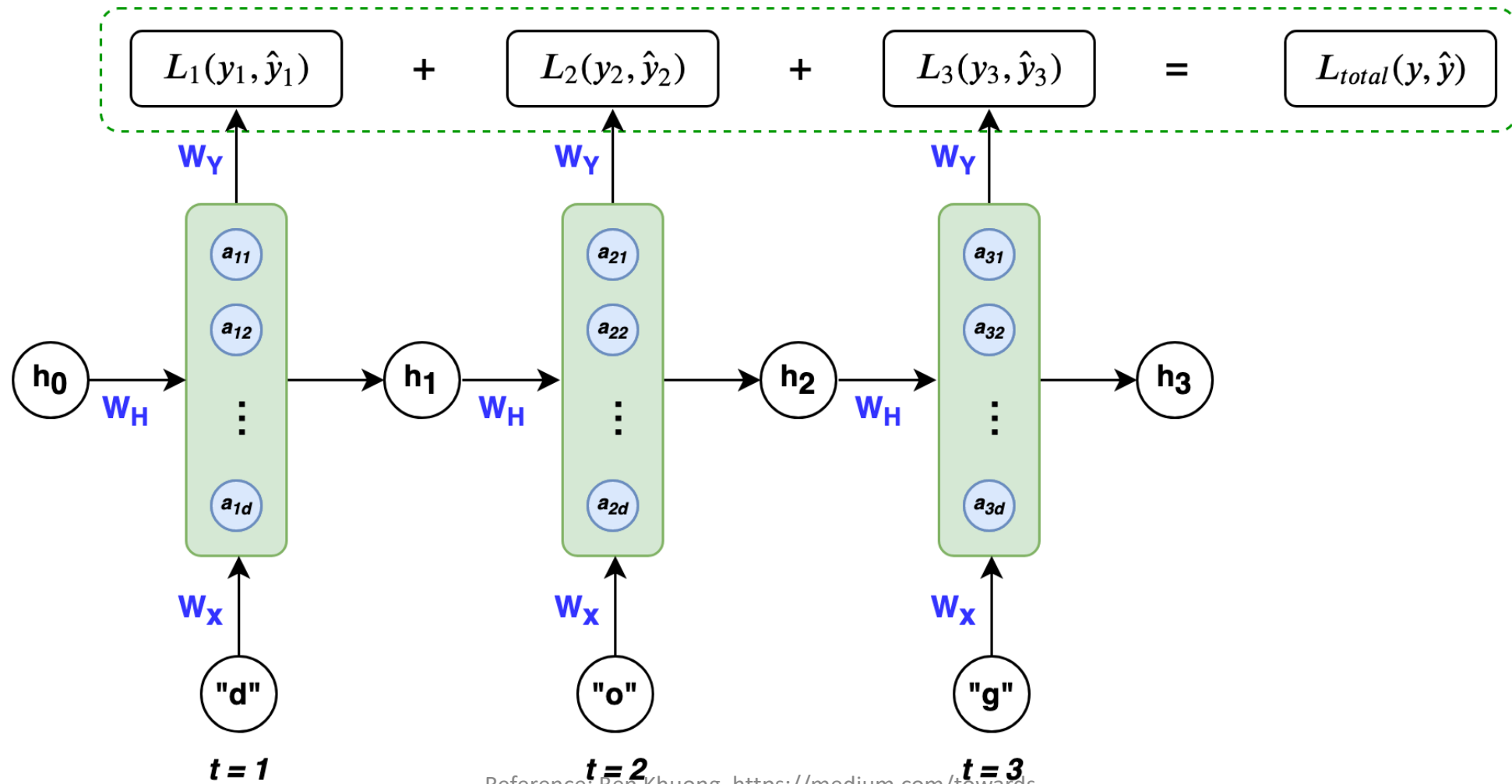
(x₁₄)

(x₁₅)

Backpropagation through time

1. Initialize weight matrices W_x , W_y , W_h randomly
2. Forward propagation to compute predictions
3. Compute the loss
4. Backpropagation to compute gradients
5. Update weights based on gradients
6. Repeat steps 2–5

Total Loss



Reference: Ben Khuong, <https://medium.com/towards-artificial-intelligence/whirlwind-tour-of-rnns-a11effb7808f>

Weight Update

$$W_i := W_i - \eta \frac{\partial L_{total}(y, \hat{y})}{\partial W_i}$$

where $i = x, y$, and h as a shorthand for the 3 weight matrices

The main idea is chain rule and to account for the loss at each time step

Function dependencies with respect to W_Y

$$L_t = -y_t \log(\hat{y}_t) \rightarrow \hat{y}_t = \text{softmax}(z_t) \rightarrow z_t = W_Y h_t$$

Chain rule with respect to W_Y

$$\frac{\partial L_t}{\partial W_Y} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial W_Y}$$

Gradient for W_Y

$$L_{total}(y, \hat{y}) = - \sum_{t=1}^n y_t \log(\hat{y}_t)$$

$$\frac{\partial L_{total}}{\partial W_Y} = \frac{\partial L_1}{\partial \hat{y}_1} \frac{\partial \hat{y}_1}{\partial z_1} \frac{\partial z_1}{\partial W_Y} + \frac{\partial L_2}{\partial \hat{y}_2} \frac{\partial \hat{y}_2}{\partial z_2} \frac{\partial z_2}{\partial W_Y} + \frac{\partial L_3}{\partial \hat{y}_3} \frac{\partial \hat{y}_3}{\partial z_3} \frac{\partial z_3}{\partial W_Y} = \sum_{t=1}^n \frac{\partial L_t}{\partial W_Y}$$

Function Dependencies with respect to W_X

$L_t = -y_t \log(\hat{y}_t) \rightarrow \hat{y}_t = softmax(z_t) \rightarrow z_t = W_Y h_t \rightarrow h_t = tanh(W_H h_{t-1} + W_X X_t) \quad (1)$

Chain rule with respect to W_x

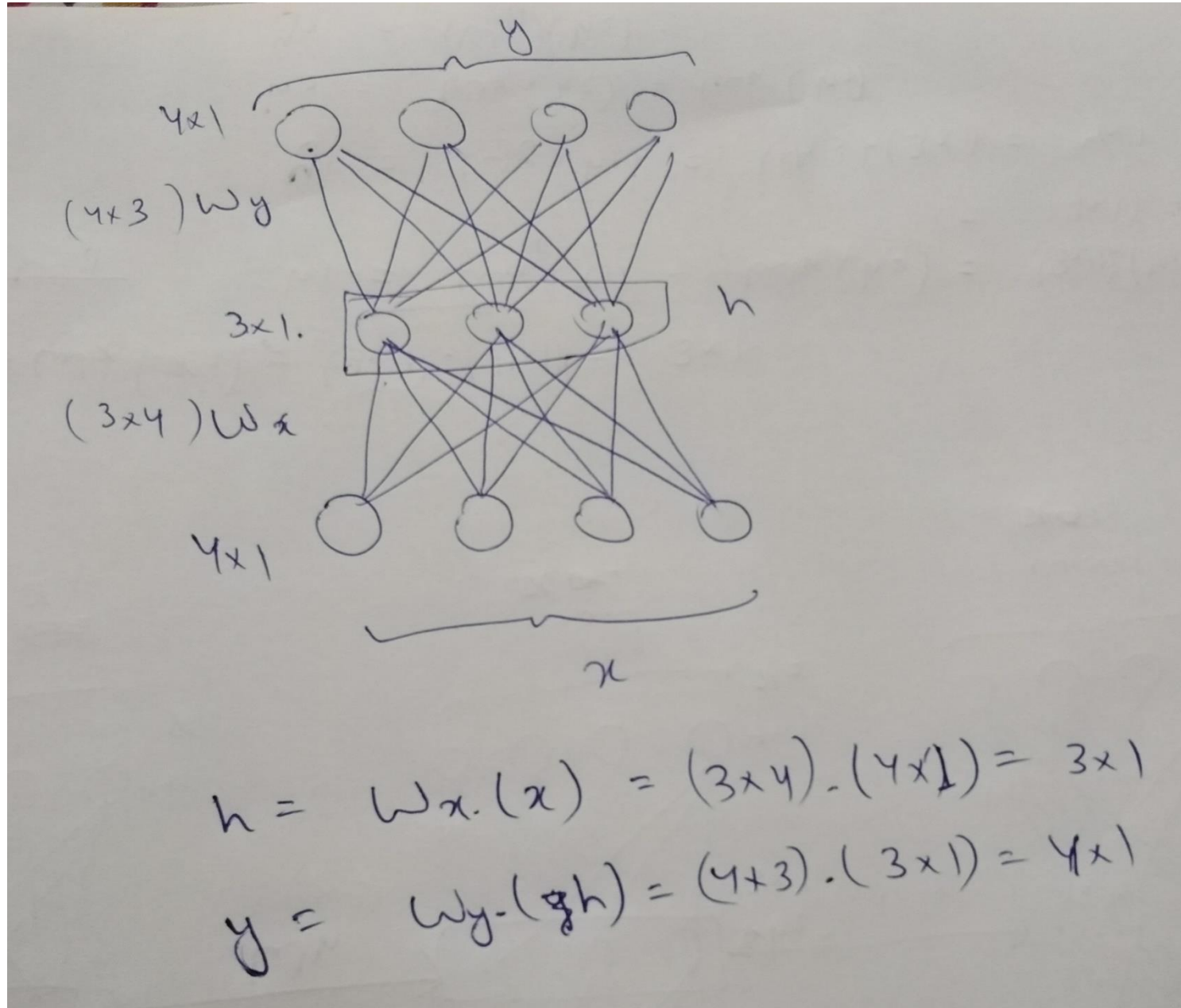
~~$\frac{\partial L_t}{\partial W_Y}$~~ $\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial W_x}$ but note, that within h_t, h_{t-1} also contains W_x , thus we need to recursively chain rule h_{t-1} until we reach h_0

~~$\frac{\partial L_t}{\partial W_Y}$~~ $\frac{\partial L_t}{\partial W_x} = \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial W_x} + \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial W_x} + \dots + \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial h_{t-n}} \frac{\partial h_{t-n}}{\partial W_x} = \sum_{k=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_x} \quad (3)$

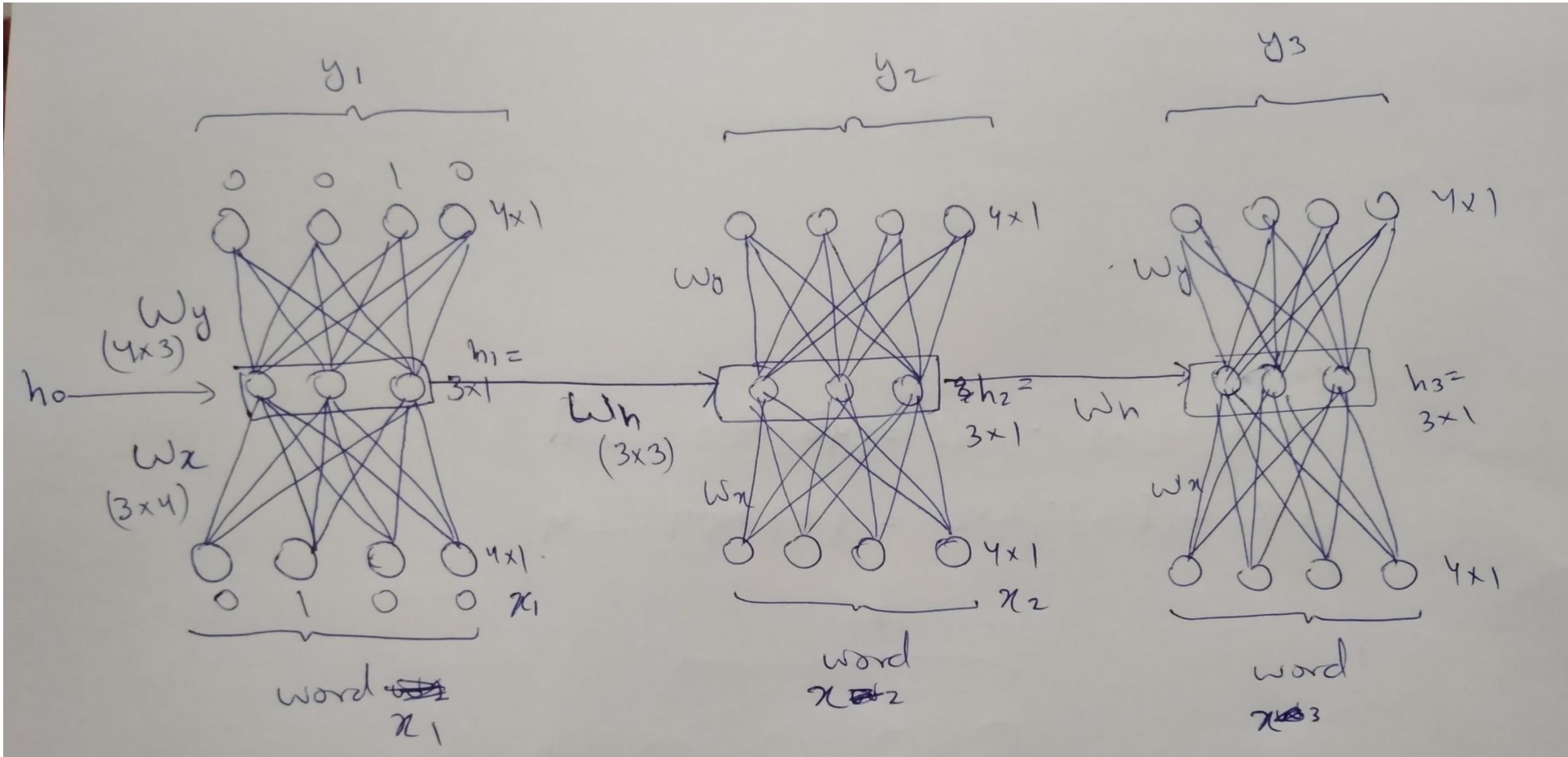
Gradient for W_X

$\frac{\partial L_{total}}{\partial W_X} = \sum_{t=1}^n \sum_{k=0}^t \frac{\partial L_t}{\partial \hat{y}_t} \frac{\partial \hat{y}_t}{\partial z_t} \frac{\partial z_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_x} \quad (4)$

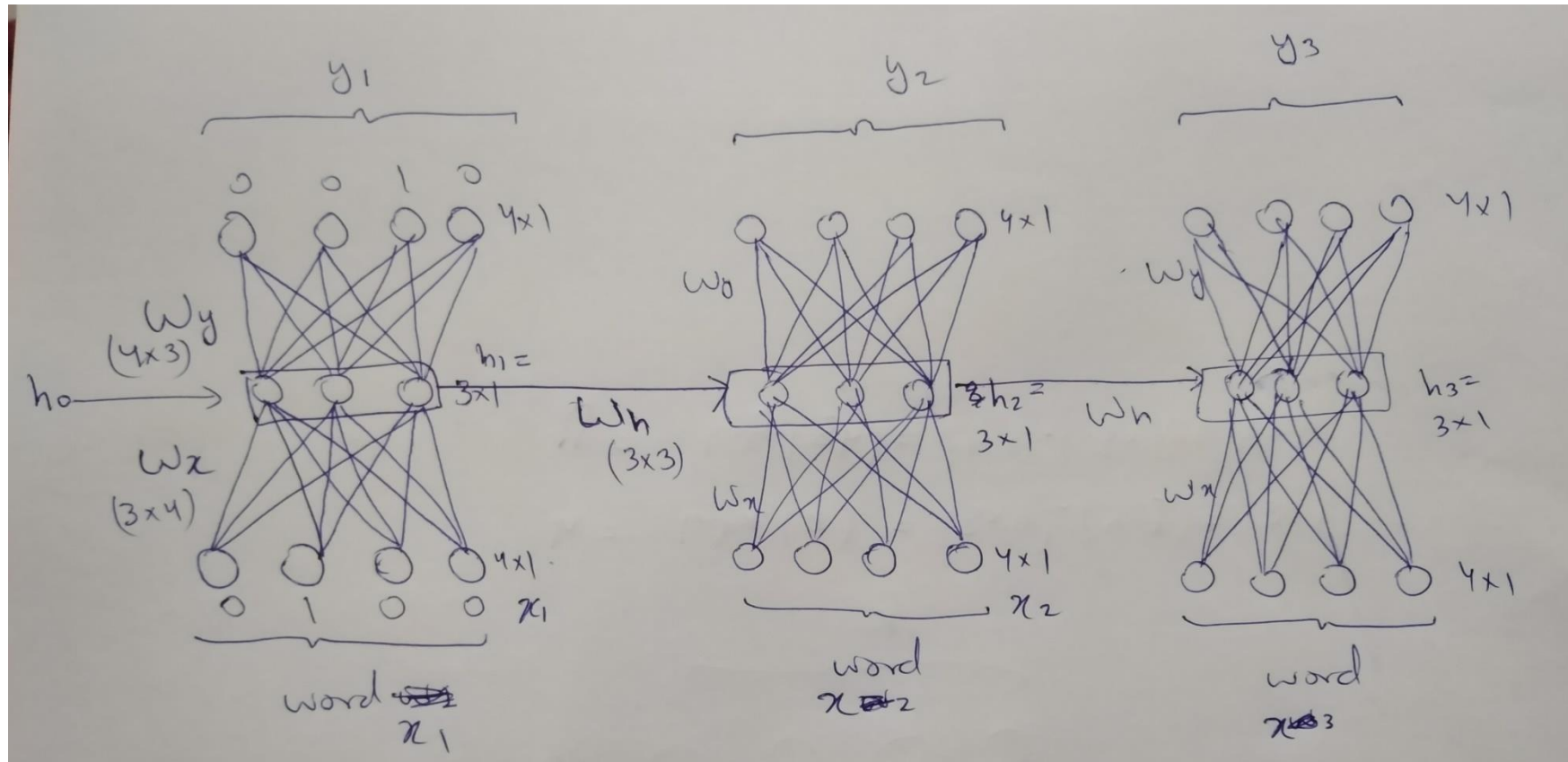
A simple neural network with 1 hidden layer



A simple recurrent neural network with 1 hidden layer



A simple recurrent neural network with 1 hidden layer



$$\begin{aligned}
 h_1 &= W_x(x_1) + W_h(h_0) = (3 \times 4)(4 \times 1) + (3 \times 3)(3 \times 1) \\
 &= (3 \times 1) + (3 \times 1) = 3 \times 1 \\
 y_1 &= W_y(h_1) = (4 \times 3)(3 \times 1) = 4 \times 1 \\
 h_2 &= W_x(x_2) + W_h(h_1) \\
 y_2 &= (W_y)(h_2) = -
 \end{aligned}$$

Reading

- Chapter 9, Speech and Language Processing. Daniel Jurafsky & James H. Martin. Third edition
<https://web.stanford.edu/~jurafsky/slp3/9.pdf>